Check for updates

International Journal of Communication Systems

WILEY

RESEARCH ARTICLE

Tail-Latency-Aware Reinforcement Learning for Dynamic Resource Allocation in 5G Network Slicing

Mahalakshmi S | Markkandan S

School of Electronics Engineering, Vellore Institute of Technology (VIT), Chennai, India

Correspondence: Markkandan S (markkandan.s@vit.ac.in)

Received: 31 March 2026 | **Revised:** 24 August 2026 | **Accepted:** 26 August 2026

Keywords: 5G and beyond | behavior cloning | network slicing | Proximal Policy Optimization (PPO) | RAN slicing | reinforcement learning | tail latency

ABSTRACT

Network slicing is a fundamental technology in 5G and beyond networks, enabling multiple virtual networks to coexist over a shared physical infrastructure while supporting heterogeneous services with diverse quality-of-service (QoS) requirements. In this context, radio access network (RAN) slicing plays a critical role in allocating radio resource units to different service slices. A fundamental challenge in RAN slicing is the dynamic allocation of radio resources under fluctuating traffic demand and user mobility, where rare but severe tail-latency events corresponding to extreme delays experienced by a small fraction of packets have a dominant impact on performance, particularly for latency-critical ultrareliable low-latency communication (URLLC) services. Although reinforcement learning has been extensively studied for RAN slicing, most existing RAN-side approaches focus on optimizing average performance metrics and permit unconstrained exploration, which can result in instability and service-level agreement (SLA) violations in safety-critical scenarios. This paper proposes a tail-latency-aware proximal policy optimization (PPO) framework for dynamic multislice RAN resource allocation. The proposed approach integrates offline behavior cloning using ns-3-generated new radio (NR) traces with online policy refinement to enable continuous adaptation. The framework incorporates a slice-aware state representation capturing traffic and mobility dynamics, an explicit tail-latency penalty targeting the 95th-percentile URLLC delay, a URLLC-aware adaptive exploration mechanism, and an SLA-driven safety layer with emergency resource reallocation. System-level evaluations across three representative scenarios demonstrate that the proposed approach consistently outperforms baseline methods in overall SLA satisfaction and tail-latency control, achieving above 92% overall SLA compliance and maintaining balanced compliance across all service slice types, particularly during sudden traffic spikes.

1 | Introduction

1.1 | Background and Motivation

By virtualizing physical resources into service-specific slices, network slicing allows many virtual networks to coexist on common infrastructure [1]. Enhanced mobile broadband (eMBB), ultrareliable low-latency communications (URLLC), massive machine-type communications (mMTC), and vehicle-to-everything (V2X) are examples of heterogeneous services that are supported by this fundamental architectural element of fifth- and sixth-generation (5G/6G) wireless systems.

The radio access network (RAN), where spectrum, power, and scheduling resources are dynamically distributed across slices, is where slicing is practically enforced, despite the fact that it is defined throughout the entire network [2]. RAN slicing is essential for enforcing service-level agreements (SLAs), especially for URLLC services where tail-latency violations predominate in perceived reliability, since these fast-timescale

© 2026 John Wiley & Sons Ltd.

All rights reserved, including rights for text and data mining and training of artificial intelligence technologies or similar technologies.

allocation decisions immediately affect buffer occupancy and scheduling delay [3].

1.2 | Limitations of Existing RAN Slicing Approaches

Reliability, resource efficiency, and isolation are traded off in RAN slicing. Strong separation between slices is provided by hard isolation, although it may result in underutilized resources. Although soft sharing enhances resource efficiency, isolation guarantees may be compromised. Dynamic resource sharing increases the likelihood of SLA violations, especially for URLLC services, even while it improves efficiency under changing traffic situations [4]. Since few delays beyond the 95th percentile can have a substantial impact on service reliability, tail latency is frequently more crucial for mission-critical applications than average delay [5]. Although GI/M/1 and other traditional queueing models provide analytical insights, they are based on stationary assumptions that do not adequately capture the bursty traffic and mobility patterns of 5G and 6G new radio (NR) networks [6]. Consequently, many existing slicing approaches based on fixed heuristics or optimization techniques struggle to adapt to rapid traffic changes, user mobility, and latency spikes in modern wireless environments [7, 8].

1.3 | Reinforcement Learning (RL) for RAN Slicing

RL has become a promising technique for adaptive RAN slicing because it can learn resource allocation strategies directly from interactions with the network environment [9]. This reduces the need for explicit analytical models. Despite this benefit, a lot of current RL techniques rely on unrestricted exploration tactics and are built to maximize average system performance. Because of this, they could result in unacceptable violations of SLAs, especially for URLLC services, which have strict reliability requirements [10]. Additionally, tail-latency targets have been integrated into RAN-side RL formulations in comparatively few studies [11]. While percentile-based reliability measurements that more accurately reflect URLLC needs receive relatively less attention, the majority of previous work focuses on average throughput or latency metrics.

Mobility-induced traffic fluctuations, scalability constraints, and issues in matching optimization goals with various quality-of-service (QoS) requirements are some of the practical obstacles that deep reinforcement learning (DRL) techniques for RAN slicing must overcome [12]. Current methods frequently rely on preset discrete action spaces or channel prediction techniques, which might limit scalability and adaptability in highly dynamic network contexts.

Under nonstationary dynamics, RAN slicing is a continuous, high-dimensional control problem [13]. Proximal policy optimization (PPO) is widely used for such tasks due to its stable, constrained policy updates [14]. It suits RAN slicing because it handles continuous actions (unlike value-based methods like DQN [15]), avoids stale data with on-policy learning [16], and enables principled exploration [17]. However, vanilla PPO lacks mechanisms to quickly respond to rare URLLC latency violations, and delayed SLA feedback with mobility can destabilize learning.

Tail-latency-aware learning has been studied in edge service scheduling [5], but prior work focuses on service placement and redundancy at second-scale granularity rather than fast-timescale, multislice RAN resource allocation under mobility. We propose a tail-latency-aware, RL framework for RAN slicing. Here, policy optimization acts as the primal process and an adaptive reliability-driven feedback parameter serves as a dual-like control signal, adjusting exploration based on SLA and tail-latency violations without an explicit Lagrangian. The scope is RAN-side dynamic slicing under mobility and traffic variability, with MAC/PHY dynamics captured via NR trace supervision. The framework integrates behavior cloning (BC) for safe initialization and online guidance and augments PPO with a tail-latency-aware reward targeting the 95th percentile URLLC delay, a URLLC-adaptive exploration controller, and a safety-aware reservation and emergency reallocation layer. A BC prior trained on realistic traces stabilizes early learning and provides corrective guidance during SLA-critical regimes, enabling safe, adaptive, and tail-latency compliant RAN slicing.

1.4 | System Architecture Overview

Figure 1 shows the end-to-end architecture of the proposed multislice RAN resource allocation framework. Heterogeneous user equipment (UE) (mobile users, V2X nodes, UAVs, IoT, and smart-home terminals) generate slice requests with QoS requirements at the service layer. These are processed through an NFV layer, where SLA-aware slice blueprints are instantiated and mapped to the 5G/6G core. The core supports dedicated and shared VNFs for flexible single and multislice services.

Slices are enforced at the RAN slicing layer, where eMBB, URLLC, mMTC, and V2X slices compete for radio resources. An RL-based control module dynamically allocates gNB resources, interacting with a learning environment that models traffic, mobility, and channel conditions. Performance feedback (SLA satisfaction, utilization, and latency) drives PPO updates, forming a closed-loop learning system.

This layered architecture separates service intent, virtualization, RAN slicing, and intelligent control, showing how SLA feedback guides real-time allocation. Details are provided in Sections 3 and 4. The remainder of this paper is organized as follows. Section 2 reviews related work on RAN slicing and learning-based resource allocation. Section 3 presents the system model and problem formulation. Section 4 details the proposed framework. Section 5 discusses experimental workflow and learning framework. Section 6 details simulation results and performance evaluation, and Section 7 concludes the paper.

2 | Related Works

2.1 | RAN Slicing and Adaptive Resource Allocation

RAN slicing supports diverse QoS requirements in 5G networks through the creation of virtualized network resources. In order to guarantee service isolation, early slicing algorithms mostly used static or optimization-based allocation

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

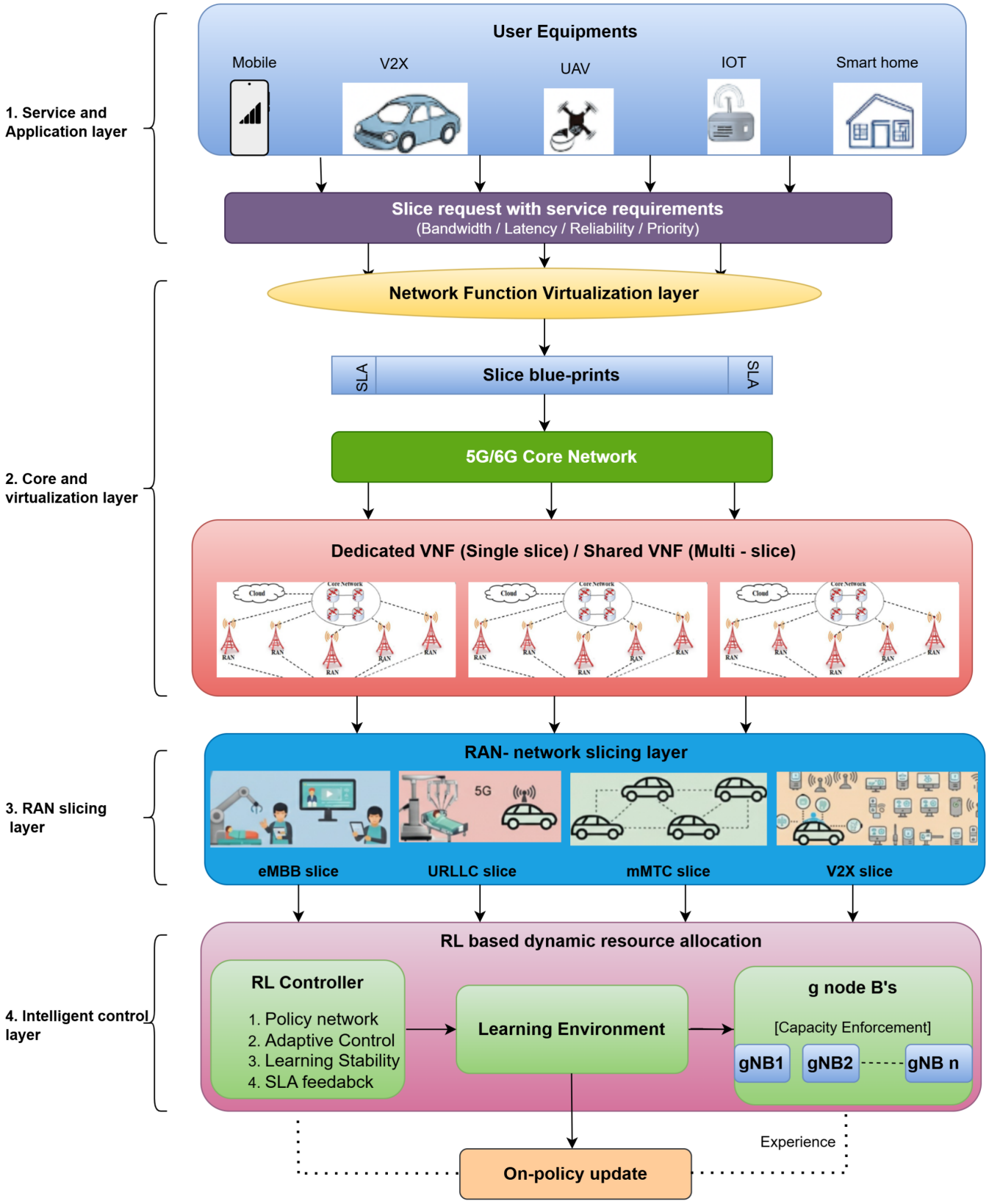


FIGURE 1 | Multislice RAN resource allocation framework.

techniques. Although these methods worked well for resource separation, they frequently produced low utilization and found it difficult to adjust to shifting network conditions [3]. Soft slicing systems were developed to overcome these restrictions by enabling regulated resource sharing between slices. Heuristic solvers are used to minimize computational complexity when using mixed-integer linear programming (MILP) to balance resource use and user satisfaction [4]. The

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

majority of solutions still rely on centralized decision-making and presumptions that might not hold true in extremely dynamic contexts, notwithstanding these advancements. Under fluctuating traffic loads, heuristic techniques like proportional and demand-aware slicing can enhance fairness [17, 18]. However, they generally react to observed conditions rather than anticipate them, and they provide limited support for strict URLLC reliability requirements, particularly those related to tail-latency performance. Latency-aware architectural studies, such as an SDN/NFV 5G slicing model analyzed via GI/M/1 queuing [6], offer theoretical insights but lack dynamic RAN-side allocation, percentile level tail-latency handling, and adaptive learning under bursty traffic and mobility. Overall, these conventional methods though insightful lack real-time adaptability to stochastic traffic, mobility, and SLA critical tail-latency, motivating learning based RAN level frameworks.

2.2 | Resource Allocation Based on DRL for RAN

DRL offers adaptive network slicing under dynamic conditions. Several studies have applied DRL to this domain, yet key gaps remain. At the network orchestration level, methods such as PPO-based PRB allocation [17], hierarchical DDQN for bandwidth and scheduling [19], and SAC-based intent-aware scheduling [9] often neglect mobility, realistic traffic patterns, or URLLC reliability. Other architectures, encompassing prediction-aided DRL [20] and frameworks such as LinkSlice [21] and THOAS [22], tackle interference or multilayer coordination but operate above the gNB control level. Similarly, O-RAN DRL [8] and MEC-oriented MARL [10] operate at slice-level abstraction without explicit user mobility modeling or percentile-based tail-latency constraints. Hybrid DDQN approaches [23] improve spectral efficiency but remain restricted to discrete action spaces. Recent studies have further expanded intelligent network slicing toward 6G-oriented architectures and advanced learning-assisted resource management. Habibi et al. [24] presented an open, intelligent, end-to-end architectural framework for network slicing in 6G communication systems, emphasizing the integration and coordination of slicing functions across network domains. Xiong et al. [25] investigated radio resource management for RAN slicing using diffusion-model and digital-twin techniques, representing another direction toward intelligent and adaptive resource allocation.

At the fine-grained RAN control layer, DRL has been applied to PRB allocation, bandwidth assignment, and computation offloading, although important limitations remain. Vehicular DQN-based resource allocation [13] employs binary offloading decisions that do not readily scale to continuous multislice optimization. Dueling DDQN improves SLA satisfaction, resource utilization, and operator profit [26], but relies on discrete bandwidth allocation without modeling mobility-induced tail-latency dynamics. LSTM-A2C captures temporal traffic variation and improves spectrum efficiency [27], yet optimizes average delay rather than high-percentile latency. Ape-X DRL enables flexible multislice resource allocation [16] through binary PRB decisions but does not explicitly address tail-latency constraints. Likewise, SDN-assisted multi-agent DQN [18] coordinates eMBB/URLLC resource allocation using delay-aware states while primarily optimizing throughput instead of latency percentiles. Beyond network slicing, recent DRL-based resource allocation studies have also considered vehicular and IoV environments. Zhang et al. [28] investigate C-V2X Mode 4 and employ DRL to adapt vehicle transmission intervals and transmission powers for the joint optimization of age of information (AoI) and energy consumption. Xie et al. [29] consider a digital-twin mobile edge network and propose a multi-agent RL approach for collaboratively allocating limited server computing resources between vehicle-twin maintenance and vehicular task processing. Gu et al. [30] develop a DRL-based resource allocation scheme for motion-blur-resistant federated self-supervised learning in IoV, where vehicle CPU frequency and transmission power are optimized to reduce energy consumption and communication/computation delay. These studies demonstrate the effectiveness of learning-based resource management in dynamic vehicular environments; however, their optimization objectives and controlled resources differ from multislice RAN allocation. Specifically, they focus on AoI and energy efficiency [28], digital-twin maintenance and edge-computing resources [29], or federated-learning training efficiency and robustness [30], rather than continuous per-gNB radio resource allocation with explicit percentile-based tail-latency control for URLLC under heterogeneous slice competition and mobility. Tail-latency awareness has received attention in edge service scheduling. TailGuard [5] and SafeTail [11] employ percentile-based objectives but operate over quasi-static resource pools at second-scale granularity, achieving reliability through request redundancy and selective replication. These approaches are fundamentally different from RAN slicing, where radio resources must be dynamically reallocated across multiple slices every few milliseconds under strict per-gNB capacity constraints and mobility-induced non-stationarity. Recent studies have also considered energy-efficient SDN/NFV control for multimedia IoV [31] and integrated vehicle routing, packet routing, and resource allocation in knowledge-defined vehicular networks [32]. These cross-layer approaches complement RAN-level slicing and indicate potential synergies for joint latency-, SLA-, and energy-aware vehicular resource management.

Nevertheless, existing DRL-based resource allocation methods largely rely on discrete actions, optimize average-delay or application-specific objectives, and do not address continuous-action, tail-latency-aware RAN slicing. The contribution of this work is therefore the integration of percentile-based objectives with on-policy RL, BC, SLA-aware safety mechanisms, and adaptive exploration under these RAN-specific constraints.

2.3 | Imitation Learning (IL) and BC in Control Systems

IL, especially BC, enables learning from expert demonstrations when rewards are hard to design [33, 34]. While BC accelerates early learning, it often fails to generalize under nonstationary conditions, motivating IL + RL hybrids for robust control.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

In networking, IL has been used for safety and stability, e.g., for digital-twin modeling [35] but not as a standalone expert that interacts online with a policy. Our framework embeds BC as a persistent, trace-driven expert that initializes and corrects a PPO policy in real time, enabling robust RAN slicing under mobility.

A two-timescale MAPPO method [7] uses expert demonstrations in vehicular slicing but operates at slice level, lacking continuous per-gNB allocation and explicit SLA shaping. Our approach performs fine grained continuous allocation and uses BC as a persistent advisor. Overall, systematic BC integration for online, SLA-aware RAN slicing especially under tail-latency constraints remains underexplored. Table 1 summarizes related learning based RAN slicing methods and contrasts them with our BC assisted PPO framework.

3 | System Model and Problem Formulation

In this section, we formalize the multislice RAN slicing problem considered in this work. The system model captures the interaction between heterogeneous service slices, spatially distributed gNBs, and an intelligent control agent that dynamically allocates radio resources. The formulation emphasizes tail-latency-aware SLA satisfaction particularly for URLLC traffic while maintaining efficient utilization across all slices.

We consider a multislice RAN composed of N gNodeBs (gNBs) serving M heterogeneous service slices, namely, eMBB, URLLC, mMTC, and V2X. Each gNB $n \in \{1, \ldots, N\}$ has a finite radio resource capacity C_n, expressed in normalized resource units. The network operates in discrete decision epochs indexed by $t = 1, 2, \ldots, T$, where each epoch corresponds to a resource allocation interval of duration.

Figure 2 illustrates the considered multislice RAN system model, where multiple gNBs jointly serve spatially distributed UEs belonging to different service slices. An intelligent agent observes aggregated network state information including load, latency, packet loss, and radio signal quality and determines slice wise resource allocation decisions. The slice gNB allocation matrix here represents an illustrative snapshot of spatial load distribution at a generic decision epoch, corresponding to a representative slice gNB association with a total of 30 UEs aggregated across all slices and gNBs.

Let $X_t(m, n) \geq 0$ denote the amount of radio resources allocated to slice m at gNB n during decision epoch t. These allocations must satisfy per-gNB capacity constraints as well as minimum slice reservation requirements. In general, the entries of $X_t(m, n)$ are time-varying and dynamically adapted by the control policy in response to evolving network conditions.

The network serves a total of K UEs with slice-specific traffic demands. A fraction $\rho_{\text{multi}} \in [0, 1]$ of UEs operate in multislice mode, simultaneously accessing multiple services, reflecting realistic 5G/6G usage scenarios.

UE mobility follows an ns-3 random waypoint model. High-mobility scenarios are characterized by average speeds of 15 m/s, whereas dense deployment scenarios use lower speeds around 5 m/s. Within the learning environment, mobility effects are abstracted as probabilistic gNB association transitions governed by a mobility probability p_{mob}, calibrated using ns-3-generated traces.

Radio propagation conditions are modeled using a 3GPP Urban Macrocell (UMa) pathloss model. A distance-based received signal strength indicator (RSSI) proxy is employed to capture large-scale channel variations. Specifically, the received signal strength at distance d from gNB n is given by

$$\text{RSSI}(d) = P_{\text{tx}} - \left[128.1 + 37.6 \log_{10}\left(\frac{d}{1000}\right)\right] + \xi, \tag{1}$$

where P_{tx} denotes the transmit power and $\xi \sim \mathcal{U}[-5, 5]$ models shadowing and measurement uncertainty. The resulting RSSI values are incorporated into the controller state as compact channel-quality indicators.

The traffic demand of slice m at gNB n during epoch t is denoted by $d_{m,n}(t)$, capturing offered load, queueing dynamics, and packet arrival processes. The average system utilization is defined as follows:

$$\bar{u}(t) = \frac{1}{N} \sum_{n=1}^{N} \frac{\sum_{m=1}^{M} X_t(m, n)}{C_n}. \tag{2}$$

This abstraction is statistically consistent to NR trace generation, and controllable learning and optimization.

The RAN slicing control issue is formulated as a constrained Markov decision process (MDP). A controller measurement of a compact system state s (t) state space, representing significant network dynamics, such as per-gNB utilization, normalized demand pressure, measurements of RSSI-based channel quality, observed latency, and a time-of-day encoding, define each decision epoch t. Given the allocation matrix, Xt(m,n), the controller selects an action S in the action set, which is consistent with the allocation.

It is aimed at determining a policy, which is expected to maximize discounted cumulative reward. The objective is to discover a policy, which maximizes the expected discounted cumulative reward, and this is formulated as follows:

$$\max_{\pi} \ \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\right], \quad \gamma \in (0, 1) \tag{3}$$

where $R(s_t, a_t)$ denotes the immediate reward explained in Section 4.2. Policy optimization is performed using PPO, as detailed in Section 4.5.

The policy operates under resource feasibility constraints including per-gNB capacity limits and minimum per-slice reservation guarantees as well as SLA reliability constraints expressed as chance constraints

$$\Pr\left(\text{SLA}_m(t) = 1\right) \geq \theta_m, \quad \forall\ m, t, \tag{4}$$

where $\text{SLA}_m(t) = 1$ indicates that slice m satisfies its key performance indicator thresholds at time t. The parameter θ_m specifies

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Comparison of related papers on learning-based resource allocation.

Ref.	Learning method	Primary optimization goal	Focus and use case	Policy/value network	Policy exploration mechanism
[17]	Independent PPO (IPPO) + random forest	Maximize operator profit under feasibility and slice constraints	End-to-end slicing (RAN + Core); public and nonpublic network slices	Policy network per agent (PPO actor–critic)	PPO stochastic exploration with RF-based feasibility filtering
[12]	AF-DRL (DQL/PPO variants) + LSTM	Maximize QoE under high mobility and slice scalability	RAN slicing; eMBB, URLLC, web browsing	Action-factorized Q-network with LSTM	ϵ-greedy with softmax action factorization
[19]	Twin-timescale dueling DDQN (TTS-DDQN)	Maximize spectrum efficiency and SLA satisfaction	Multislice RAN with MAC scheduling	Dueling Q-network (value-based)	ϵ-greedy with reward clipping and traffic-aware sampling
[9]	Soft actor–critic (SAC)	Fulfill slice intents (latency, packet loss) and maximize cumulative reward	RAN radio resource scheduling; eMBB, URLLC	Actor–critic networks	Entropy-regularized stochastic exploration
[7]	Multi-agent PPO with expert demonstrations	Joint spectral and energy efficiency	V2X slicing	Multi-agent actor–critic	PPO with expert guidance
[20]	Trust-region policy with Lyapunov DRL	Long-term reward maximization with constraint stability	RAN slicing	Actor–critic networks	Trust-region policy updates
[10]	MAPPO/PPO-based MARL	Maximize resource efficiency under QoS constraints	End-to-end slicing (RAN + MEC); eMBB, URLLC	Actor–critic networks	Entropy-regularized PPO
[23]	DDQN-PER with PtrNet-LSTM	Balance SLA satisfaction and QoE while improving spectrum efficiency	RAN slicing; eMBB, URLLC	DDQN with pointer network and LSTM	ϵ-greedy with prioritized experience replay
[13]	DQN	Improve resource utilization at the edge	Joint edge, RAN, and transport networks; IoV and smart cities	Deep Q-network with experience replay	ϵ-greedy
[26]	Dueling GAN-DDQN	Maximize operator profit and resource utilization	RAN; VoLTE, video streaming, URLLC	Q-networks augmented with GAN-based discriminator	ϵ-greedy
[27]	LSTM-A2C	Maximize spectral efficiency, profit, and SLA satisfaction	RAN slicing under mobility; VoLTE, eMBB, URLLC	Recurrent actor–critic networks	Softmax-based action selection
[16]	DDQN	Minimize allocated PRBs while satisfying QoE	Multislice RAN	Apex-style DDQN architecture	ϵ-greedy
[18]	DQN	Maximize achievable data rate	RAN; eMBB and URLLC	Online and target Q-networks with replay memory	ϵ-greedy

(Continues)

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | (Continued)

Ref.	Learning method	Primary optimization goal	Focus and use case	Policy/value network	Policy exploration mechanism
[28]	MPDQN (multipass DQN)	Minimize weighted AoI and communication energy consumption	C-V2X Mode-4 V2V communication; RRI and transmission-power allocation; multi-priority queues and NOMA	MPDQN for hybrid discrete RRI and continuous transmission-power actions	ϵ-greedy for discrete-action exploration with continuous-action perturbation
[29]	TMTPRCS (MADRL)	Maximize computing-resource utility while satisfying delay constraints	Digital Twin Mobile Edge Network; joint computing-resource allocation for twin maintenance and vehicular task processing	Centralized-training and distributed-execution actor–critic networks	Adaptive ϵ-greedy exploration
[30]	DRL-BFSSL (SAC)	Minimize energy consumption and delay	Federated SSL in IoV; motion-blur-resistant aggregation with CPU-frequency and transmission-power allocation	SAC actor with twin critic and target critic networks	Entropy-regularized stochastic exploration

the minimum acceptable SLA satisfaction probability, with URLLC typically requiring $\theta_{\text{URLLC}} \approx 0.99$. The probability is taken over stochastic traffic arrivals, UE mobility, channel fluctuations, and injected network shock events.

4 | Methodology

We consider a multislice RAN composed of M service slices (eMBB, URLLC, mMTC, and V2X) and N gNBs. Time is slotted, and at each decision epoch t, the controller allocates radio resources across slices and gNBs under dynamic traffic, mobility, and channel conditions.

Figure 3 illustrates the general construction of the proposed tail-latency conscious, SLA-based RL framework of dynamic slice of RAN. The controller has a closed-loop operation in which the measured network state (where slice demand, latency, and packet loss, RSSI, utilization, mobility, and multislice interactions are collected) is entered into a policy network that is implemented as CNN and LSTM modules. The CNN identifies spatial correlations with the slice-gNB grid of resources, and the LSTM identifies the time sequence of the traffic and latency.

The initialization and regularization of the policy output is then done with a BC prior, trained on the trace of the NR ns-3, and hard constraints and temporal smoothing of the capacity per-gNB and the minimum reservation imposed. In addition to the assignment of the key resources, the adaptive SLA-conscious control layer will be in charge to track the URLLC reliability and tail-latency figures as well. The dynamic adaptive exploration controller is a type of dynamically adjusting the magnitude of exploration of the policy by dynamically varying the parameter of the policy, the β and trades off the learning with the operational safety. In the event of tail-latency spikes or SLA degradation being observed, then a tail-aware boost and emergency reallocation mechanism is activated, which temporarily assigns importance to significant slices in order to limit the unusual observed adverse delay outliers.

The surrogate environment emulates the dynamics of the actual RAN such as the aggregation of demand of the UEs, the load shift when changing mobility and the contention between the various slices and gives real-time feedback of the performance. In order to update policies in a fashion that allows a stable learning process under immensely nonstationary network conditions without discontinuing SLA, we apply on-policy PPO with a clipped objective and generalized advantage estimation (GAE) to update the policies.

4.1 | Resource Model and State Representation

Let $X_t(m,n)$ denote the amount of radio resources allocated to slice $m \in \{1,\ldots,M\}$ at gNB $n \in \{1,\ldots,N\}$ during time step t. Each gNB n has a finite capacity C_n, imposing the per-gNB feasibility constraint

$$\sum_{m=1}^{M} X_t(m,n) \le C_n, \ \forall n. \tag{5}$$

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

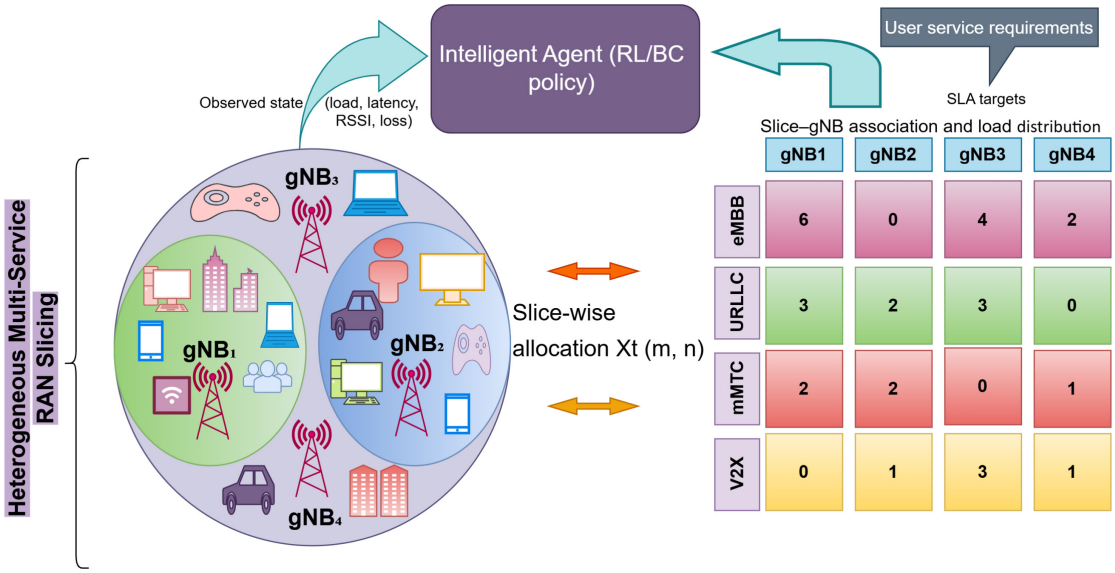


FIGURE 2 | System model of multislice RAN.

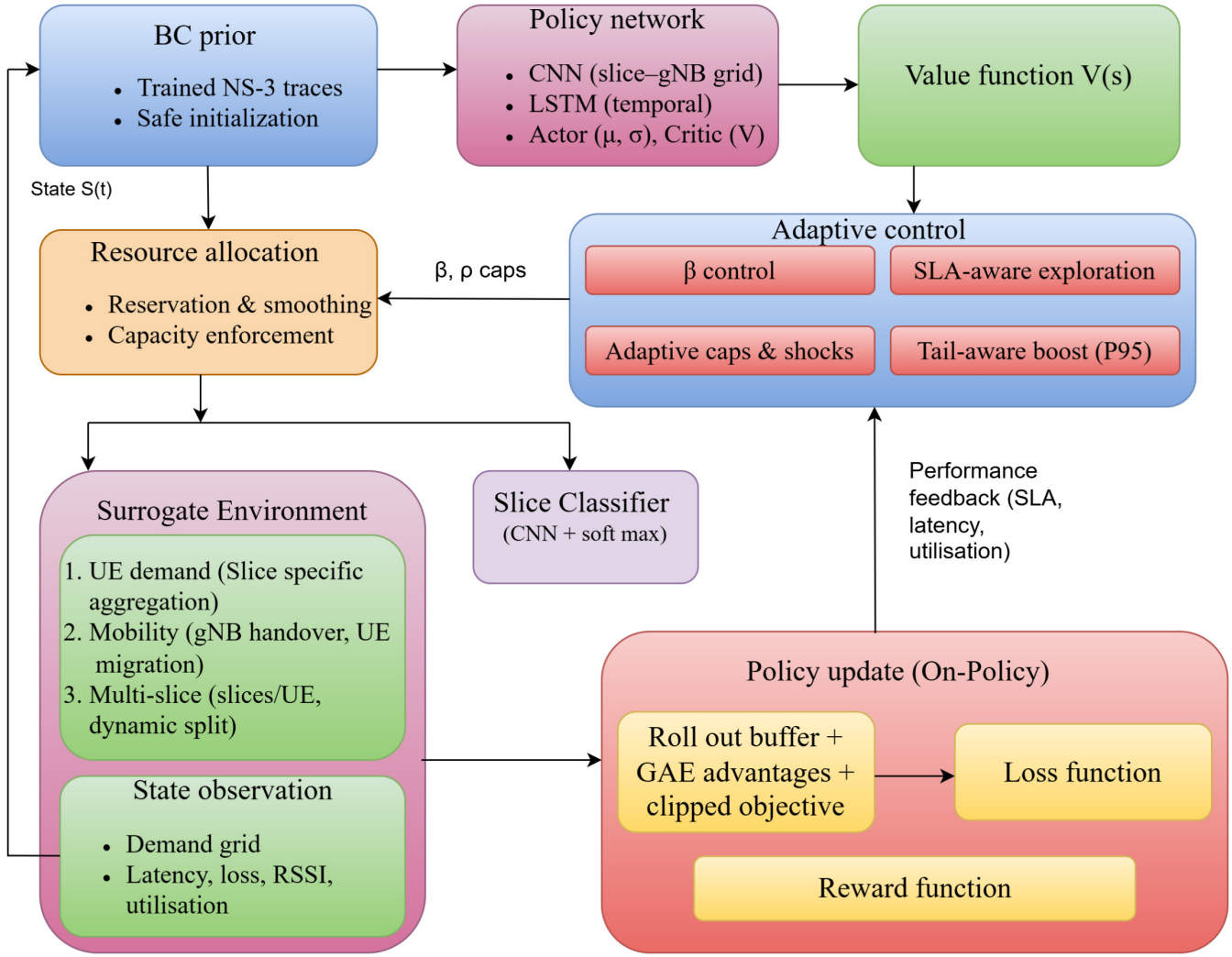


FIGURE 3 | Overall structure of the proposed work.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

For the purpose of service continuity, each slice is associated with a minimum safe reservation floor $\mu_m^{\min}(n)$, such that

$$X_t(m,n) \geq \mu_m^{\min}(n), \ \forall m,n. \tag{6}$$

These constraints are enforced explicitly in the environment after every policy action, as implemented in the SLA enforcement and per-gNB normalization stages of the code. The system state s_t aggregates radio, traffic, and temporal information, including normalized per-slice demand pressure at each gNB, per-gNB utilization ratios, latency, packet loss, RSSI indicators, and time-of-day features. The policy can take into consideration temporal variations brought on by user movement and traffic fluctuations, as well as spatial interactions between gNBs and slices, the state representation that combines both past observations and current network conditions. A CNN-LSTM architecture is used to take use of these features. While the LSTM layers learn temporal connections related to mobility, bursty traffic, and delayed SLA implications, the convolutional layers extract spatial patterns from the demand grid. Tail latency is used as the major dependability parameter rather than average latency since URLLC systems are extremely sensitive to extreme delay occurrences. The observed latency at gNB n at time t is represented by $L_{t,n}$. The tail latency can be defined as follows:

$$L_t^{95} = \text{Percentile}_{95}\left(\{L_{t,n}\}_{n=1}^{N}\right). \tag{7}$$

Tail latency is a more reliable indicator of rare but critical service degradation events, particularly under high-mobility and shock conditions. Let $D_{m,t} = \sum_n d_{m,n}(t)$ denote the aggregate traffic demand of slice m at time t. The slice-level allocation efficiency is defined as follows:

$$\eta_{m,t} = \frac{\sum_n X_t(m,n)}{D_{m,t}}. \tag{8}$$

When $\eta_{m,t}$ deviates significantly from unity, a slice-dependent penalty is applied to prevent inefficient resource utilization. Penalties for over-allocation and under-allocation are asymmetrical, with noncritical slices facing worse penalties in the presence of URLLC demand. This mechanism aligns well with the efficiency constraints and scaling logic implemented in the code.

4.2 | Tail-Latency-Driven Reward Design

Under limited observation windows, percentile-95 strikes a balance between statistical stability and sensitivity to uncommon occurrences. The instantaneous reward R_t is designed to maintain efficiency while giving priority on URLLC dependability. The URLLC-specific reward component is defined as follows:

$$R_t^{\text{URLLC}} = \begin{cases} r_1, & L_t^{95} \leq 10 \text{ ms}, \\ r_2, & 10 < L_t^{95} \leq 12 \text{ ms}, \\ -\kappa\left(L_t^{95} - 12\right), & L_t^{95} > 12 \text{ ms}, \end{cases} \tag{9}$$

where $r_1 > r_2 > 0$ and $\kappa > 0$ control the severity of tail-latency violations. Efficiency-aware penalties R_t^{eff}, SLA satisfaction rewards R_t^{SLA}, and utilization incentives R_t^{util} are also included in the URLLC-specific reward component. A URLLC-dominant convex combination is used to calculate the ultimate reward.

$$R_t = 0.55R_t^{\text{URLLC}} + 0.25R_t^{\text{eff}} + 0.15R_t^{\text{SLA}} + 0.05R_t^{\text{util}}. \tag{10}$$

In this case, R_t^{eff} penalizes excessive and inadequate allocation in relation to demand, R_t^{SLA} rewards aggregate SLA satisfaction across all slices, and R_t^{util} promotes effective gNB utilization. Because URLLC traffic is safety-critical, this weighting purposefully biases learning toward tail-latency stability. Moderate changes in these weights do not change qualitative performance patterns, according to sensitivity analysis.

Particularly when observations are gathered over brief time periods, using the instantaneous 95th-percentile latency as a reward signal may create variability. By explicitly targeting extreme outcomes or modeling return distributions, more advanced techniques like Conditional Value-at-Risk (CVaR)-based optimization and distributional RL offer a principled treatment of tail behavior. However, this work's percentile-based incentive was selected for pragmatic reasons. It aligns directly with the SLA metrics commonly used by network operators, where the 95th-percentile latency serves as a standard indicator of URLLC performance. Second, the URLLC-dominant convex weighting in (10) combined with the BC regularization stabilizes the variance of the composite signal during training.

4.3 | URLLC-Aware β Adaptation

Instead of updating the dual variable using gradient ascent, the exploration–conservatism trade-off parameter β_t is formulated as a piecewise feedback control law governed by observed URLLC performance. Let S_t^U indicate the moving average URLLC SLA satisfaction, and let u_t denote the system utilization. The β-update rule is established as follows:

$$\beta_{t+1} = \begin{cases} 0.85, & S_t^U < 80 \ \wedge \ \text{no crisis}, \\ \text{Sched}(t), & t \in \{100, 250, 500+\}, \\ \text{clip}(\beta_t + \Delta^+, 0.25, 0.80), & S_t^U < 95, \\ \text{clip}(\beta_t - \Delta^-, 0.25, 0.80), & S_t^U > 99 \ \wedge \ u_t < 0.5, \\ \beta_t, & \text{otherwise}, \end{cases} \tag{11}$$

where $\Delta^+ = 0.05$ and $\Delta^- = 0.02$. The operator $\wedge$ denotes logical conjunction. A crisis state represents sustained URLLC SLA degradation below 80%, during which strong corrective exploration is triggered and maintained until recovery to avoid oscillatory behavior. The bounds 0.25–0.80 establish a safe exploration envelope that maintain adaptability under shock events and mobility while blocking excessive stochasticity. The function $\text{Sched}(t)$ refers to a rollout indexed decay schedule that gradually diminishes exploration as training progresses.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The above logic of adaptation of β is applicable to various operating regimes of URLLC to provide efficient resource allocation. In case URLLC SLA satisfaction declines to a critical level, an emergency mode is enabled, which causes an immediate switch to a conservative behavior and avoidance of lengthy tail-latency violations. This emergency state is sustained until recovery to eliminate oscillatory behavior due to noisy SLA observations. Beyond emergency situations, the exploration is diminished by a predetermined decay process over time at predetermined training phases to aid stable convergence of policy across long learning times. In the case where the performance of URLLC has been impaired in a way that is not critical, but it is being affected, incremental increase in β is applied to facilitate remedial action without provoking the sudden intervention. On the other hand, in case of constant high URLLC reliability and a general low use of the system, the β is minimized to eliminate conservatism to reclaim resource efficiency of noncritical slices. At all other points, the value of β t is fixed, which adds hysteresis to the control algorithm and eliminates extraneous oscillations between successive decision intervals. Combined, these rules permit rapid reaction to severe reliability deterioration, effortless adjustment to normal operation, and effective utilization under constant conditions, without making use of gradient-based dual updates, which are vulnerable to slow and inaccurate SLA feedback.

The bounded operating envelope [0.25, 0.80], the crisis-mode hysteresis that prevents oscillatory switching under noisy SLA observations, and the direct closed-loop coupling to observed SLA outcomes rather than to noisy constraint-violation gradients are the three structural properties that provide empirical stability even though the β adaptation rule (11) is not derived from a formal optimization framework like Lagrangian dual updates or constrained policy optimization (CPO). Conventional methods use gradient ascent to update dual variables in response to constraint violations. However, because SLA input is frequently scarce and delayed in highly dynamic RAN contexts, such updates may become unstable when URLLC performance deteriorates. Instead, the suggested closed-loop β adaptation is based on SLA results that are immediately observed. This prevents needless modifications and enables the controller to respond swiftly when reliability declines.

TABLE 2 | β-adaptation and policy convergence stability over 3000 training episodes (high-mobility scenario).

Metric	Min	Max	Mean	Std
β (exploration parameter)	0.33	0.40	0.34	0.013
Policy entropy $H(\pi)$	0.62	0.78	0.72	0.029
Policy log-std	-0.80	-0.64	-0.70	0.034

The evolution of β and policy convergence over 3000 training episodes in the high-mobility scenario are shown in Table 2. During training, β stays between 0.33 and 0.40, which is substantially inside the permitted interval of [0.25, 0.80] specified in (11). Instead of functioning as a fixed parameter, a brief rise to 0.40 shows that the controller reacts to SLA degradation as necessary. The relatively steady URLLC conditions seen throughout training and the use of modest update steps are consistent with the little variance in β. At the same time, the wider admissible range preserves sufficient flexibility to accommodate more challenging operating conditions. Policy entropy and log-standard-deviation remain in convergent ranges, indicating stable behavior without collapse or divergence. A formal comparison against Lagrangian-PPO and CPO baselines is reserved for future work.

To assess the sensitivity of the proposed framework's performance to the manually chosen thresholds (80%, 95%, and 99%) and step sizes ($\Delta^+ = 0.05,\ \Delta^- = 0.02$) in the β-adaptation rule (11), we conduct a one-at-a-time sensitivity analysis in the high-mobility scenario, reported in Table 3. Varying the SLA thresholds across a wider operating range (75/92/98 to 85/97/99.5) shifts the average converged β from 0.30 to 0.39, with URLLC SLA varying within 64.9%–68.4% and overall SLA within 90.7%–92.2%. Similarly, varying the step sizes by a factor of four (Δ^+/Δ^- from 0.025/0.01 to 0.10/0.04) shifts URLLC SLA within 66.3%–67.8% and overall SLA within 91.5%–91.9%. Across all tested configurations, P95 latency remains within 4.5–4.8 ms. These results indicate that the framework's performance is not overly sensitive to the specific threshold and step-size values chosen, with the default configuration achieving the most favorable overall SLA and tail latency among the tested settings, whereas neighboring configurations yield only modest performance variation rather than instability or degradation.

4.4 | Safety-Aware Reservation and Emergency Reallocation

URLLC reliability violations constitute safety-critical events with low tolerance for delayed correction. Depending solely on RL to rediscover emergency behavior results in catastrophic SLA collapse.

TABLE 3 | Sensitivity of the proposed framework to β-adaptation thresholds and step sizes (high-mobility scenario).

Parameter	Value	Avg. β	URLLC SLA (%)	Overall SLA (%)	P95 latency (ms)
Thresholds (%)	75/92/98 (threshold-low)	0.39	68.4	91.8	4.7
	80/95/99 (default)	0.34	67.2	92.2	4.5
	85/97/99.5 (threshold-high)	0.30	64.9	90.7	4.8
Step size (Δ^+, Δ^-)	0.025/0.01 (step-small)	0.32	66.3	91.9	4.6
	0.10/0.04 (step-large)	0.37	67.8	91.5	4.7

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Therefore, explicit safety reallocation is applied as a safety margin, whereas RL improves performance within the safety margins. When URLLC SLA is less than a critical level, the emergency reservation enforcement is triggered. In this case, minimum URLLC reservations are increased as follows:

$$X_t^{\mathrm{URLLC}}(n) \geq \mu_{\mathrm{URLLC}}^{\mathrm{emg}}(n) \tag{12}$$

where $\mu_{\mathrm{URLLC}}^{\mathrm{emg}}(n)$ denotes an emergency minimum reservation that satisfies $\mu_{\mathrm{URLLC}}^{\mathrm{emg}}(n) > \mu_{\mathrm{URLLC}}^{\min}(n)$ and ensures URLLC protection during critical conditions. If required, resources are transferred from nonemergency slices:

$$X_t(j,n) \leftarrow X_t(j,n) - \delta_n, \quad X_t(\mathrm{URLLC},n) \leftarrow X_t(\mathrm{URLLC},n) + \delta_n, \tag{13}$$

where $j \in \{\mathrm{mMTC}, \mathrm{V2X}\}$. This process is directly proportional to the critical reallocation logic. The interaction between the emergency reallocation mechanism in (12)–(13) and the RL policy is governed by three structural properties. First, the emergency reallocation is invoked only when URLLC SLA falls below a strict critical threshold, ensuring that the mechanism is reserved for transient reliability degradation rather than continuous policy correction. Second, the safety layer operates as a hard constraint applied after policy action sampling and does not directly inject gradients into policy learning; the agent learns from rewards observed under the constraint-enforced allocation, encouraging policies that maintain URLLC reliability through their own decisions rather than relying on the safety mechanism. Third, the safety layer is structurally distinct from the policy network: emergency reallocation is invoked only as a corrective step after the policy output, ensuring that the policy retains full responsibility for steady-state allocation decisions, whereas the safety mechanism serves as a bounded fallback during transient SLA degradation.

4.5 | Learning Framework

This framework uses PPO as the main on-policy RL algorithm because it is stable in spaces where actions are continuous and works well in networks that are not always stable. PPO is the main optimization tool, and the proposed tail-latency-aware reward design, safety constraints, and URLLC-aware control are all external control layers that change how exploration works, how resources are allocated, and how risk is perceived without changing the core PPO mechanism. The policy is trained using the clipped surrogate objective

$$\mathscr{L}^{\mathrm{PPO}} = \mathbb{E}\left[\min\left(\rho_t A_t,\ \mathrm{clip}\left(\rho_t, 1-\epsilon, 1+\epsilon\right)A_t\right)\right], \tag{14}$$

where ρ_t denotes the probability ratio between the updated and previous policies and A_t represents the generalized advantage estimate. Here, the PPO probability ratio is defined as follows:

$$\rho_t = \frac{\pi_\theta\left(a_t|s_t\right)}{\pi_{\theta_{\mathrm{old}}}\left(a_t|s_t\right)}, \tag{15}$$

which constrains policy updates by limiting deviation from the previous policy through clipping, ensuring stable on-policy learning under nonstationary RAN dynamics. Entropy regularization is employed to encourage sufficient exploration, whereas an adaptive BC regularization term is incorporated during early training stages to stabilize learning and prevent unsafe exploratory actions.

Algorithm 1 summarizes the proposed tail-latency-aware RL framework for multislice RAN resource allocation. The actor network π_θ and the critic (value) network are initialized together with a safe allocation baseline defined by the minimum reservation guarantees $\{\mu_m^{\min}(n)\}$. An adaptive BC scheduler is employed during early training stages to provide safe initialization and prevent unstable exploratory actions under stringent URLLC constraints. During each training rollout, the environment is reset along with SLA statistics and shock indicators. The system state s_t, which records aggregate traffic pressure, per-gNB utilization, delay statistics, and packet loss indicators, is observed by the agent at each decision epoch t. By calculating the 95th-percentile tail latency L_t^{95}, as defined in (7), the URLLC dependability condition is explicitly monitored. The closed-loop control law in (11) is used to update the exploration–conservatism parameter β_t based on the observed SLA satisfaction and system utilization. The policy then determines the approximate slice-wise resource allocation by sampling a continuous action $a_t \sim \pi_\theta(s_t;\beta_t)$. When URLLC or V2X tail latency above predetermined levels, a tail-slice boost is implemented to guarantee reliability under latency violations. Subsequently, efficiency constraints and minimum reservation guarantees are enforced using (6), followed by an emergency URLLC reallocation step when critical SLA degradation is detected, as described in (12) and (13). The resulting allocation is normalized to satisfy per-gNB capacity constraints in (5) before being executed in the environment. After executing the allocation, the composite reward R_t is computed according to (10), incorporating URLLC-dominant tail-latency penalties, efficiency terms, SLA satisfaction, and utilization incentives. The observed transition (s_t, a_t, R_t, s_{t+1}) is stored for on-policy learning. At the end of each episode, the policy parameters are updated using the PPO clipped surrogate objective in (14), with GAE providing low-variance advantage signals. This recursive process continues until convergence and results in a slicing policy, which achieves a combined benefit of efficient utilization, tail-latency stability and SLA satisfaction.

4.6 | Adaptive BC Weight Scheduler

The BC regularization weight w_t used in Algorithm 1 is adapted online through a piecewise time-based decay combined with a URLLC-SLA-history-driven correction. Let $p = t/T$ denote training progress, where t is the current episode and T is the total number of training rollouts. Let $\bar{s}_t$ denote the URLLC SLA satisfaction averaged over the last H episodes:

$$\bar{s}_t = \frac{1}{H}\sum_{i=1}^{H} s_{t-i}^{\mathrm{URLLC}}, \tag{16}$$

where $s_t^{\mathrm{URLLC}} \in [0,1]$ is the URLLC SLA satisfaction at episode t. The BC weight is computed as follows:

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$$w_t = \mathrm{clip}\big(w_{\mathrm{base}}(p) + \Delta(\bar{s}_t),\ 0.05,\ 0.80\big), \tag{17}$$

where $w_{\mathrm{base}}(p)$ follows a five-phase decay scaled by initial weight w_0:

$$w_{\mathrm{base}}(p) = \begin{cases} w_0 & p < 0.10, \\ 0.75\, w_0 & 0.10 \le p < 0.20, \\ 0.50\, w_0 & 0.20 \le p < 0.40, \\ 0.33\, w_0 & 0.40 \le p < 0.60, \\ 0.17\, w_0 & p \ge 0.60. \end{cases} \tag{18}$$

The SLA-driven correction $\Delta(\bar{s}_t)$ is scaled by a maximum boost $\Delta_{\max}$:

$$\Delta(\bar{s}_t) = \begin{cases} +\Delta_{\max} & \bar{s}_t < 0.70, \\ +0.71\ \Delta_{\max} & 0.70 \le \bar{s}_t < 0.85, \\ +0.29\ \Delta_{\max} & 0.85 \le \bar{s}_t < 0.95, \\ 0 & 0.95 \le \bar{s}_t \le 0.99, \\ -0.43\ \Delta_{\max} & \bar{s}_t > 0.99. \end{cases} \tag{19}$$

The default configuration uses $w_0 = 0.60$, $\Delta_{\max} = 0.35$, and $H = 10$. While the clipping guarantees that $w_t \in [0.05, 0.80]$ during training, the piecewise structure offers restricted BC influence with proportional sensitivity to URLLC degradation. Table 4 (Section 6) reports the framework's sensitivity to various scheduler parameters. The engineering decisions that encode standard URLLC operating regimes are reflected in the piecewise constants (decay phase boundaries 0.10, 0.20, 0.40, and 0.60 and SLA tier thresholds 0.70, 0.85, 0.95, and 0.99). It also provides an empirical characterization of the framework's robustness to moderate variation in the three primary parameters w_0, $\Delta_{\max}$, and H.

TABLE 4 | Hyperparameters for simulations.

Component	Value used
Model type	On-policy actor–critic
Policy type	Gaussian stochastic policy
Actor network (policy)	Shared backbone + MLP head (linear layer to actions)
Critic network (value)	Shared backbone + MLP head (linear layer to value)
LSTM hidden size	128
Episode length	128 steps
Rollout batch size	128 steps per policy update
PPO minibatch size	64 (per epoch)
Optimizer	Adam
Discount factor (γ)	0.995
Nonlinearity	ReLU
Entropy regularization	Learned log-standard deviation, decayed over training
Learning rate (policy + value)	3×10^{-4} (shared)
Reward scaling	1

5 | Experimental Workflow and Learning Framework

A trace-based approach is embraced to balance realism and learning efficiency, which combines high fidelity NR simulations with a low weight learning environment. The ns-3 simulator is utilized in the creation of offline traces to trace the fine-grained PHY/MAC-layer dynamics, and in online learning is utilized as a statistically representative abstraction that preserves the required couplings between traffic load, latency, mobility, utilization and packet loss. Subframe PHY/MAC effects such as Rayleigh fast fading, HARQ retransmissions, and MAC scheduler contention are abstracted into aggregate distributions calibrated against the ns-3 traces rather than simulated explicitly in the online loop. Three principal operational rules are considered. In the high-mobility scenario, 30 UEs representing vehicular users with frequent handovers follow a random waypoint mobility model with a mean speed of 15 m/s. In the learning environment, this behavior is abstracted through a UE association transition probability of 0.40 per time step. In the dense deployment scenario, 80 UEs with moderate mobility (mean speed 5 m/s) emulate congestion-dominated operation, mirrored in the learning environment using the same UE count and a reduced mobility probability of 0.20, and 60% of users concurrently create traffic for various service slices in the multislice scenario, which uses 50 UEs with modest mobility to capture cross-slice coupling effects. This behavior is directly mirrored in the learning environment.

A four-gNB topology with a 3GPP UMa propagation model, proportional-fair scheduling, queueing dynamics, adaptive MCS selection, retransmissions, and mobility-induced variability is used to generate ns-3 traces in all scenarios. Large-scale on-policy training is made possible while maintaining statistical consistency by mapping continuous mobility dynamics in ns-3 to probabilistic association transitions in the surrogate environment.

A distance-based RSSI proxy in line with the 3GPP UMa pathloss model is used to capture channel variability without disclosing the entire PHY-layer information. Performance statistics are collected over fixed-length windows of duration T_w to obtain stable learning features from ns-3 traces. In particular, the window-averaged throughput is computed as follows:

$$\mathrm{Thr}_w = \frac{8B_w}{T_w \times 10^6}\ \mathrm{Mbps}, \tag{20}$$

where B_w denotes the total number of successfully received bytes within the window. These window-level measurements, together with latency and packet loss statistics, form the supervised dataset used to train the BC prior. The proposed framework follows a two stage learning pipeline that integrates offline

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Algorithm 1. Tail-latency-aware RL-based resource allocation.

Require: gNB capacities $\{C_n\}$, minimum slice reservations $\{\mu_m^{\min}(n)\}$, SLA targets

Ensure: Resource allocation matrix $X_t(m,n)$

Initialize $X_0(m,n) \leftarrow \mu_m^{\min}(n)$
Initialize policy network π_θ, value network V_ψ
Initialize exploration parameter β_0
Initialize adaptive behavior-cloning (BC) weight scheduler based on training progress and recent URLLC SLA history
for each training rollout **do**
  Reset environment, SLA statistics, and shock indicators
  for each time step t **do**
    Observe system state s_t (traffic pressure, utilization, latency, loss)
    Compute URLLC tail latency L_t^{95} using (7)
    Update exploration parameter β_t using control law (11)
    Sample action $a_t \sim \pi_\theta(s_t;\beta_t)$
    if URLLC or V2X SLA violation is detected **then**
      Apply tail-slice priority boost
    end if
    Enforce minimum reservations and efficiency constraints using (6)
    if emergency URLLC reallocation is required **then**
      Apply emergency reallocation using (12)–(13)
    end if
    Normalize allocations to satisfy gNB capacity constraint (5)
    Execute allocation X_t in the environment
    Compute reward R_t using (10)
    Store transition (s_t, a_t, R_t, s_{t+1})
  end for
  Update policy parameters using PPO objective (14)
end for
return learned policy π_θ

IL with online RL to achieve stable and adaptive RAN slicing control.

In the offline stage, a BC policy is trained in a supervised manner using episodic NR traces. Each trace record consists of a compact network state representation paired with a slice wise resource allocation selected by a rule based oracle scheduler. To improve robustness and generalization, the BC model operates on a low-dimensional state comprising aggregate offered load, aggregate throughput, mean latency, a 95th percentile latency proxy, packet drops, and mean RSSI. Training uses a 70/15/15 train–validation–test split, after which the BC policy is frozen and retained as a safe prior.

In the online stage, a PPO agent interacts with a Python-based surrogate environment that preserves essential couplings between utilization, latency, throughput, and packet loss while enabling efficient on-policy updates. To prevent unsafe exploration during early training, the PPO objective is augmented with a BC regularization term whose weight w_t decays gradually as learning progresses; the precise scheduler is specified in Section 4.6.

Training is conducted over 3000 episodes, corresponding to approximately 3.84×10^5 environment time steps. Resource allocation is performed in normalized capacity units, with the per-gNB capacity vector modeling inter site heterogeneity. To

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

evaluate robustness, stochastic congestion shock events are injected with nonzero probability per episode, representing bursty traffic arrivals, sudden mobility changes, and transient load spikes.

6 | Performance Evaluation and Simulation Results

6.1 | Performance Metrics and Evaluation Criteria

Slice-level SLA compliance is assessed at each decision interval using QoS requirements specific to each slice. For the eMBB slice, the SLA is considered satisfied when the achieved throughput exceeds 0.9 of the offered traffic demand, ensuring efficient support for high-data-rate services. For the URLLC slice, stricter performance requirements apply: The 95th-percentile latency must remain below 12 ms, and packet delivery reliability must be maintained above 99. The mMTC slice SLA is defined by massive access robustness, requiring packet loss rate below 5%. For V2X services, which are latency critical but less stringent than URLLC, the SLA is satisfied when the 95th-percentile latency does not exceed 20 ms. To facilitate unified reporting across slices, we define a binary slice level SLA indicator for slice n at time t as

$$\mathbb{I}_t^{(n)} = \begin{cases} 1, & \text{if slice } n \text{ satisfies its SLA at time } t, \\ 0, & \text{otherwise,} \end{cases} \tag{21}$$

and compute the overall SLA satisfaction as the average of slice level indicators,

$$\mathrm{SLA}_t^{\mathrm{overall}} = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}_t^{(n)}. \tag{22}$$

While SLA satisfaction captures service compliance, it does not fully reflect how efficiently radio resources are utilized. Let $A_t(n)$ denote the total radio resources allocated to slice n at time t, and let $Q_t(n)$ denote the corresponding aggregate traffic demand of slice n at time t. Using these quantities, we quantify allocation mismatch through over allocation and under allocation indices. The over allocation index is defined as follows:

$$\Omega_t = \frac{\sum_{n=1}^{N} \max\left(0,\ A_t(n) - Q_t(n)\right)}{\sum_{n=1}^{N} A_t(n)}, \tag{23}$$

which measures the fraction of allocated resources that remain unused due to conservative or inefficient allocation. Conversely, the under allocation index is defined as follows:

$$\Upsilon_t = \frac{\sum_{n=1}^{N} \max\left(0,\ Q_t(n) - A_t(n)\right)}{\sum_{n=1}^{N} Q_t(n)}, \tag{24}$$

which captures the proportion of unmet demand and directly reflects service starvation risk. Both indices are averaged over time to provide stable performance indicators.

To evaluate how effectively allocated resources are converted into useful service, we define a slice-level efficiency metric. Let $\mathrm{ServedTraffic}_t(n)$ denote the successfully delivered traffic of slice n during time interval t. The efficiency of slice n is computed as follows:

$$\varepsilon_t^{(n)} = \frac{\mathrm{ServedTraffic}_t(n)}{A_t(n)}. \tag{25}$$

Latency is a critical performance indicator for RAN slicing, particularly for URLLC and V2X services where tail behavior dominates user experience. In addition to percentile based latency metrics, we report the average system latency. Let $D_t^{(u)}$ denote the packet delay experienced by user u during decision interval t, and let $\mathcal{U}_t$ denote the set of active users at time t. The average latency is computed as follows:

$$\overline{D}_t = \frac{1}{|\mathcal{U}_t|} \sum_{u \in \mathcal{U}_t} D_t^{(u)}. \tag{26}$$

All reported latency metrics are averaged over evaluation episodes. To evaluate convergence behavior and learning stability, we analyze RL return statistics and policy dispersion. Let R_t denote the composite reward defined in Section IV-E. For an episode of length T, the episode return is as follows:

$$G = \sum_{t=1}^{T} R_t, \tag{27}$$

and the average return across evaluation episodes is computed as $\mathbb{E}[G]$. Higher average returns indicate improved policy quality and better alignment with the multiobjective reward design.

Policy stability is evaluated using the dispersion of the learned action distribution. For a stochastic policy $\pi_\theta(a_t|s_t)$, the policy entropy is defined as follows:

$$\mathscr{H}(\pi_\theta) = \mathbb{E}_{a_t \sim \pi_\theta}\left[-\log \pi_\theta(a_t|s_t)\right]. \tag{28}$$

For continuous action spaces parameterized by a Gaussian distribution, the policy standard deviation is tracked via the learned log-standard deviation parameter,

$$\sigma_t = \exp\left(\log \sigma_t\right). \tag{29}$$

Excessively low values may indicate premature convergence, but lower entropy and decreased action variance throughout training indicate policy convergence. Instead than evaluating raw performance, these indicators are utilized to evaluate learning stability. We examine policy behavior during shock events and transitory overload scenarios, such as bursty traffic, mobility shifts, or load spikes, to assess robustness under challenging circumstances. Every episode logs a binary shock indicator $\mathbb{S}_t \in 0, 1$, where $\mathbb{S}_t = 1$ indicates a shock at time t. When $\mathbb{S}_t = 1$, robustness is evaluated by looking at latency metrics and SLA satisfaction. In contrast to static heuristic approaches, the suggested learning-based strategy maintains stronger SLA compliance during shocks by dynamically adjusting its policy and exploration in reaction to SLA degradation. The hyperparameter values utilized during the learning phase are listed in Table 4.

Four heuristic baselines are considered for comparison: equal slicing, demand-aware slicing, delay-aware slicing, and

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

channel-aware slicing [12]. These rule-based techniques provide reference points for assessing learning-based resource allocation methods and are evaluated under the same operational conditions as the proposed framework. In addition to these heuristics, baseline PPO, Lagrangian-PPO, and TTS-DDQN [19] are included as learning-based comparison methods. TTS-DDQN provides a stronger recent value-based baseline and is adapted to the same simulation environment, traffic conditions, slice definitions, resource constraints, and scenario settings used for the other learning-based methods. Its dueling-DDQN decision mechanism and ϵ-greedy exploration are retained.

Let R_{total} denote the total available radio resources at a decision epoch, and let N be the number of network slices. The resource allocation to slice n at time t, denoted by $A_t(n)$, is computed as follows.

Equal slicing: This baseline allocates resources uniformly across all slices, independent of traffic demand, latency, or channel conditions,

$$A_t(n) = \frac{R_{\text{total}}}{N}, \ \forall n. \tag{30}$$

Demand-aware slicing: Resources are allocated in proportion to the aggregate traffic demand $Q_t(n)$ of each slice,

$$A_t(n) = \frac{Q_t(n)}{\sum_{i=1}^{N} Q_t(i)} R_{\text{total}}. \tag{31}$$

Delay-aware slicing: This scheme prioritizes slices experiencing higher latency, allocating resources proportionally to the observed delay indicator $L_t(n)$,

$$A_t(n) = \frac{L_t(n)}{\sum_{i=1}^{N} L_t(i)} R_{\text{total}}. \tag{32}$$

Channel-aware slicing: To compensate for poor radio conditions, this heuristic allocates resources inversely proportional to the spectral efficiency $\eta_t(n)$,

$$A_t(n) = \frac{\eta_t(n)^{-1}}{\sum_{i=1}^{N} \eta_t(i)^{-1}} R_{\text{total}}. \tag{33}$$

These heuristic schemes operate independently at each decision epoch without learning, memory, or cross-episode adaptation and therefore provide deterministic rule-based reference points for comparison with the learning-based controllers.

To ensure that the comparison is not biased by degenerate baseline behavior, the four heuristic baselines, baseline PPO, and Lagrangian-PPO incorporate a minimum eMBB allocation floor of 5% of the per-gNB capacity. This common feasibility constraint prevents zero-allocation outcomes when an allocation rule drives the eMBB share toward zero. TTS-DDQN employs its own discrete action formulation, in which a minimum 10% slice share is imposed during action-space construction, and therefore does not use the 5% eMBB floor applied to the preceding baselines. The proposed framework additionally employs its full per-slice minimum reservation layer (6) and dynamic emergency reallocation mechanism (12) and (13); these safety mechanisms are unique components of the proposed framework and are not applied to TTS-DDQN or the other baseline algorithms.

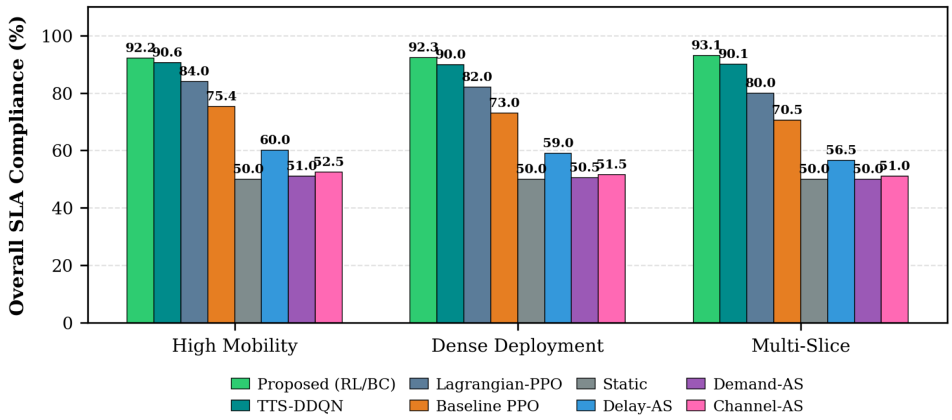
6.2 | Simulation Results and Discussion

We implement the proposed framework in Python 3.12 with a PyTorch backend, with the device fixed to CPU in all experiments. All metrics are computed using 100-episode windowed statistics over 3000 training episodes, reflecting long term behavior rather than instantaneous fluctuations. Performance plots are generated using the Matplotlib Python library. As a reference, we implement a clipped PPO baseline [14] with Gaussian policies, the standard clipped surrogate objective, generalized advantage estimation (GAE(λ)), entropy regularization, and gradient-norm clipping. The actor–critic baseline uses the same CNN + LSTM encoder as the proposed method to ensure architectural fairness. Unless otherwise stated, the PPO baseline uses a policy learning rate of 2×10^{-5}, discount factor $\gamma = 0.997$, GAE parameter $\lambda = 0.92$, clipping parameter $\epsilon_{\text{clip}} = 0.20$, three PPO epochs per update, a minibatch size of 64, and a rollout length of 128 steps per update matching the training horizon used in our main experiments. All baselines share the same reward shaping, SLA targets, and environment interface so that performance differences can be attributed to the resource allocation policy.

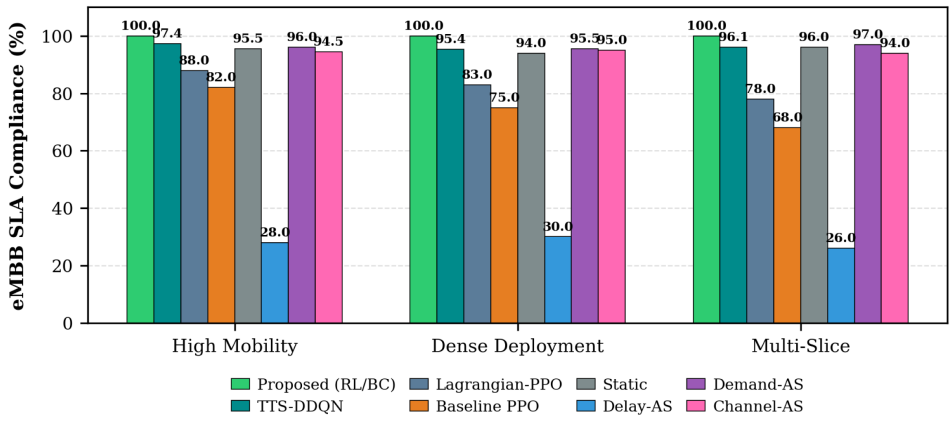
Figure 4a shows that the proposed framework achieves the highest overall SLA satisfaction across all scenarios, reaching 92.2%, 92.3%, and 93.1% in the high-mobility, dense-deployment, and multislice scenarios, respectively. The TTS-DDQN baseline provides the closest learning-based comparison, achieving 90.58%, 89.95%, and 90.11%, respectively. Although these results demonstrate the effectiveness of value-based resource adaptation, TTS-DDQN remains approximately 1.6–3.0 percentage points below the proposed framework across the three scenarios. The Lagrangian-PPO baseline attains 80.0%–84.0%, narrowing the gap relative to vanilla PPO (70.5%–75.4%) but remaining below both TTS-DDQN and the proposed framework. Among the heuristic schemes, Delay-AS achieves moderate overall SLA (56%–60%) due to its prioritization of latency-sensitive slices, whereas Static, Demand-AS, and Channel-AS converge to near-symmetric values around 50%–53%, reflecting their limited ability to adapt across heterogeneous slice requirements.

As shown in Figure 4b, the proposed framework achieves 100% eMBB SLA across all scenarios, jointly maintaining broadband service requirements while protecting latency-sensitive traffic. TTS-DDQN also achieves strong eMBB performance, with SLA satisfaction of 97.38%, 95.41%, and 96.06% in the high-mobility, dense-deployment, and multislice scenarios, respectively, but remains below the proposed framework in all three cases. Heuristic methods such as Static, Demand-AS, and Channel-AS also achieve high eMBB SLA (94%–97%) because their proportional demand- or channel-oriented allocation tends to favor broadband traffic. However, their overall SLA and URLLC results indicate that high eMBB satisfaction alone does not provide balanced cross-slice performance. Delay-AS, in contrast, sacrifices eMBB substantially (26%–30%) by reallocating resources

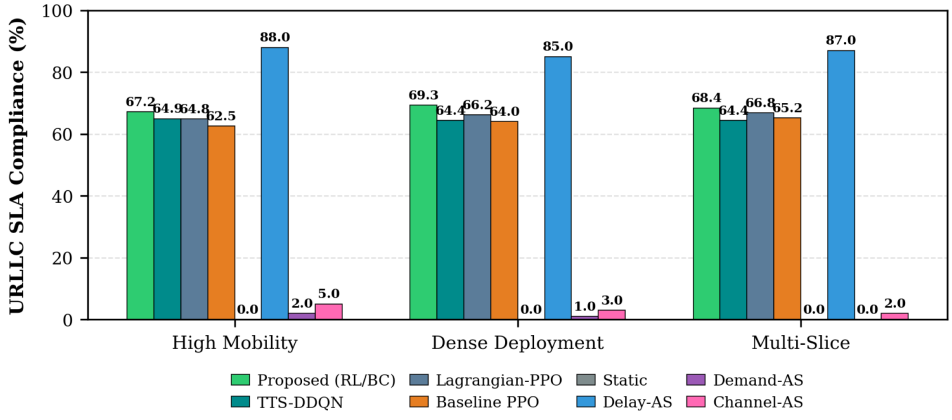
10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



(a) Overall SLA



(b) eMBB SLA



(c) URLLC SLA

FIGURE 4 | Service level agreement.

toward latency-stressed slices. The Lagrangian-PPO baseline exhibits a similar, though less severe, trade-off: Its dual-variable penalty on URLLC violation reduces eMBB SLA to 78%–88%. Vanilla PPO maintains eMBB SLA in the 68%–82% range, with further degradation under dense and multislice conditions. The results therefore show that TTS-DDQN provides a considerably stronger eMBB baseline than the PPO variants, whereas the proposed framework maintains complete eMBB SLA satisfaction together with higher overall SLA performance.

Figure 4c reports URLLC SLA across the three scenarios. The proposed framework achieves 67.2%, 69.3%, and 68.4% URLLC SLA in the high-mobility, dense-deployment, and multislice scenarios, respectively, while simultaneously maintaining 100% eMBB SLA. TTS-DDQN achieves 64.94%, 64.40%, and 64.37%, respectively, showing competitive URLLC performance but remaining below the proposed framework in all three scenarios. The Lagrangian-PPO baseline achieves 64.8%–66.8% URLLC SLA, whereas vanilla PPO reaches approximately 62%–65%. The

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

relative ordering between TTS-DDQN and Lagrangian-PPO varies slightly across scenarios, which reflects their different control objectives: Lagrangian-PPO explicitly penalizes URLLC constraint violations, whereas TTS-DDQN uses value-based resource adaptation without an explicit tail-latency constraint. Delay-AS achieves the highest URLLC SLA (85%–88%) by aggressively prioritizing latency-stressed slices, but this improvement comes at a substantial cost to eMBB SLA, as shown in Figure 4b. The remaining heuristic schemes (Static, Demand-AS, and Channel-AS) provide negligible URLLC SLA satisfaction (0%–5%). Overall, the results indicate that the proposed framework provides the strongest cross-slice balance, achieving the highest overall SLA and URLLC performance among the learning-based methods while preserving 100% eMBB SLA. The mMTC and V2X slices remain SLA-saturated across the evaluated schemes and scenarios; therefore, subsequent analysis focuses primarily on eMBB and URLLC, where the slicing decisions have the greatest impact.

The simulation recorded 231 shock periods across the scenarios, covering 1334 episodes, with a mean shock duration of 5.77 episodes (minimum = 1, maximum = 28). These shock periods represent both short- and longer-duration stress conditions in the RAN slicing environment. Since shock robustness is evaluated as a property of the learned controllers, the rule-based heuristic schemes are not included in this analysis.

Figure 5 compares the 95th-percentile latency across scenarios. The proposed method consistently achieves the lowest P95 latency among the learning-based methods, with values of 4.2–4.5 ms, indicating effective control of tail delay. TTS-DDQN records P95 latency of 5.57, 6.18, and 5.86 ms in the high-mobility, dense-deployment, and multislice scenarios, respectively. Although TTS-DDQN achieves competitive overall SLA performance, its higher P95 latency indicates that value-based resource adaptation without an explicit tail-latency objective is less effective at suppressing upper-tail delay. The Lagrangian-PPO baseline achieves P95 latency of 4.5–5.0 ms, close to the vanilla PPO baseline (4.5–5.1 ms), reflecting that the dual-variable mechanism primarily enforces the URLLC constraint rather than directly optimizing the latency tail. Delay-AS achieves the lowest P95 latency among the heuristic methods (10.5–11.5 ms) because of its latency-prioritizing allocation rule. The remaining heuristic schemes (Static, Demand-AS, and Channel-AS) exhibit higher P95 latency of 13.6–15.6 ms, reflecting their limited ability to control queue buildup under contention. Overall, the results show that the proposed framework maintains the strongest tail-latency performance across the three representative scenarios.

Figure 6 reports the over-allocation index across scenarios. The proposed method achieves only 1%–3% over-allocation, indicating tight regulation of excess resource allocation. In contrast, TTS-DDQN records 59.99%, 62.84%, and 61.67% over-allocation in the high-mobility, dense-deployment, and multislice scenarios, respectively. This result is particularly important when interpreted together with the low under-allocation of TTS-DDQN (0.92%–1.69% in Table 5: The value-based controller avoids resource shortages largely through comparatively aggressive provisioning, whereas the proposed framework maintains very low over-allocation while achieving higher overall SLA compliance and lower tail latency. The Lagrangian-PPO baseline exhibits 43%–48% over-allocation, approximately half that of vanilla PPO (92%–97%) but still substantially higher than the proposed framework. Static slicing shows 0% over-allocation by construction because its rigid equal-share allocation does not adapt to instantaneous demand. Demand-AS exhibits moderate over-allocation of 24%–28% by approximately tracking demand, whereas Delay-AS, Channel-AS, and Baseline PPO exhibit high over-allocation of 84%–99%. Taken together with the under-allocation results in Table 5, these findings show that low under-allocation alone does not imply efficient resource regulation, and that the proposed framework achieves a more controlled balance between resource provisioning, SLA satisfaction, and tail-latency protection.

Figure 7 shows the evolution of SLA satisfaction over time together with the identified congestion-shock periods. The proposed framework maintains a comparatively stable SLA trajectory during both normal operation and prolonged shock

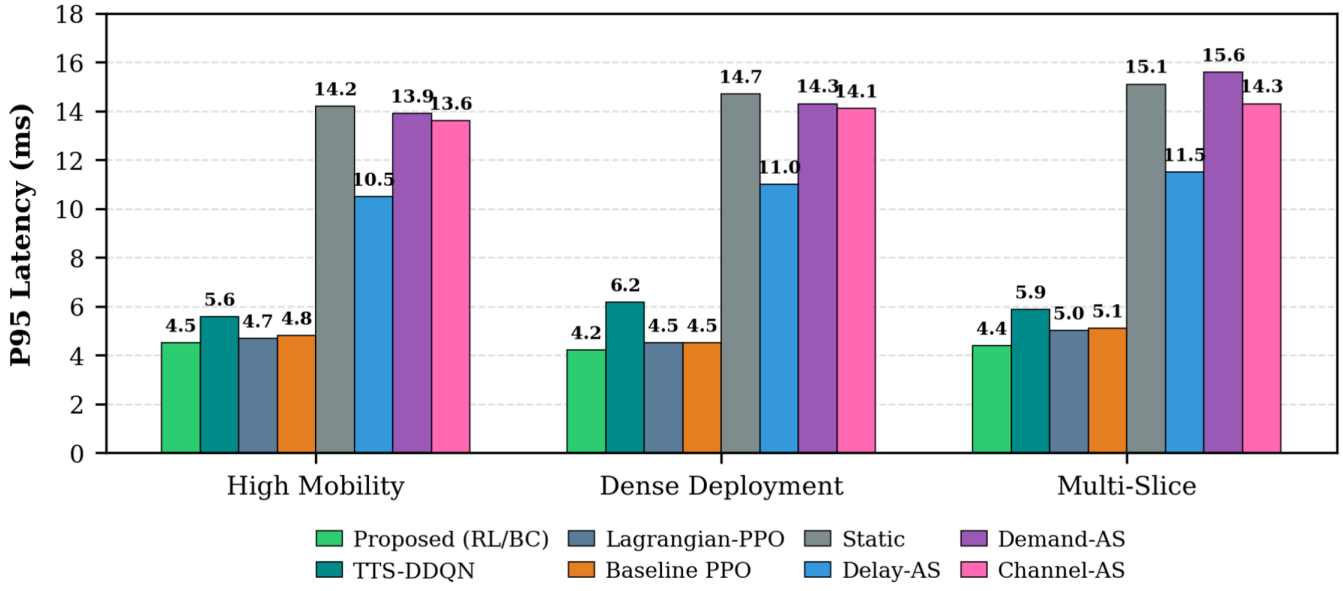


FIGURE 5 | P95 latency.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

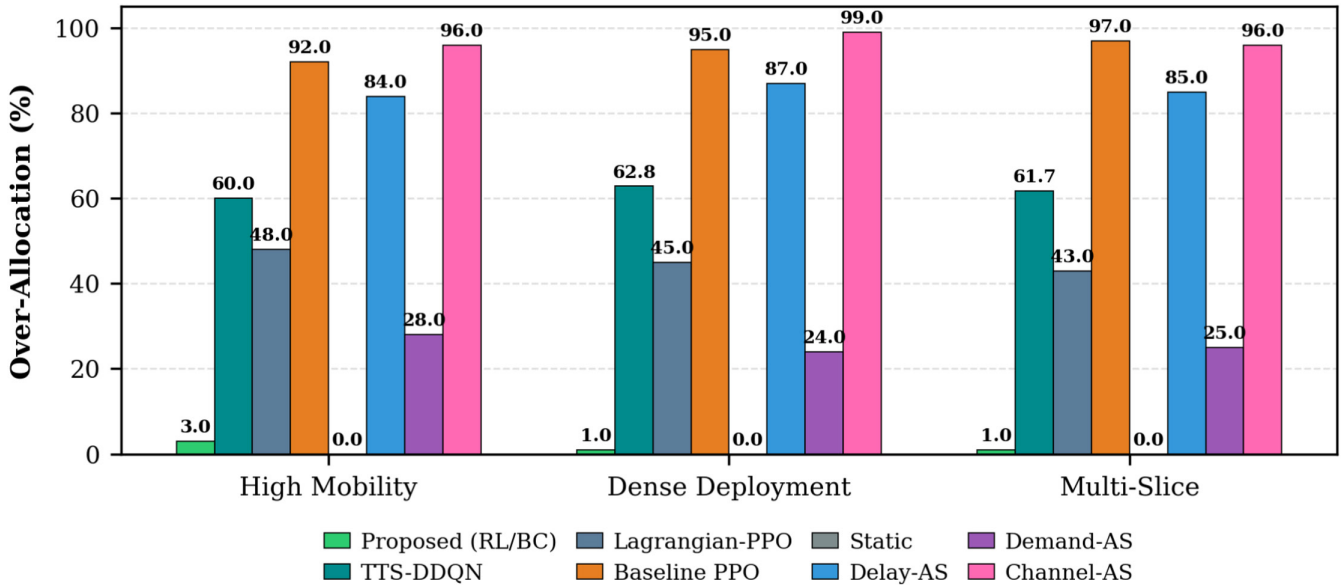


FIGURE 6 | Over-allocation.

TABLE 5 | Comparative performance metrics across representative scenarios.

Scenario	Algorithm	Avg. decision time (s)	Avg. P99 latency (ms)	Avg. policy log-std	Avg. under-allocation (%)	Avg. mMTC loss rate (%)
Scenario 1 (high mobility)	Static	0.0032	13.77	0	2.00	6.00
	Delay-AS	0.0029	13.20	0	4.20	6.00
	Demand-AS	0.0034	13.90	0	2.80	6.00
	Channel-AS	0.0033	13.60	0	3.00	6.00
	Baseline PPO	0.0045	5.20	0.1954	22.00	1.25
	Lagrangian-PPO	0.0048	4.90	0.1200	17.00	1.22
	TTS-DDQN	0.0100	7.22	N/A	0.92	2.63
	Proposed	0.0038	4.53	−0.6899	28.19	2.09
Scenario 2 (dense deployment)	Static	0.0033	14.04	0	2.00	6.00
	Delay-AS	0.0031	13.40	0	4.30	6.00
	Demand-AS	0.0036	14.30	0	2.90	6.00
	Channel-AS	0.0035	14.10	0	3.10	6.00
	Baseline PPO	0.0052	4.90	0.1954	24.00	1.25
	Lagrangian-PPO	0.0055	4.70	0.1200	19.00	1.24
	TTS-DDQN	0.0101	7.78	N/A	1.12	2.80
	Proposed	0.0042	4.24	−0.6909	58.41	1.91
Scenario 3 (multislice)	Static	0.0034	14.21	0	2.10	6.00
	Delay-AS	0.0032	13.80	0	4.40	6.00
	Demand-AS	0.0038	15.60	0	3.00	6.00
	Channel-AS	0.0036	14.30	0	3.20	6.00
	Baseline PPO	0.0060	5.50	0.1954	26.00	1.25
	Lagrangian-PPO	0.0062	5.20	0.1200	21.00	1.26
	TTS-DDQN	0.0114	7.55	N/A	1.69	2.71
	Proposed	0.0045	4.48	−0.6907	47.45	2.02

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



FIGURE 7 | Performance during shock events.

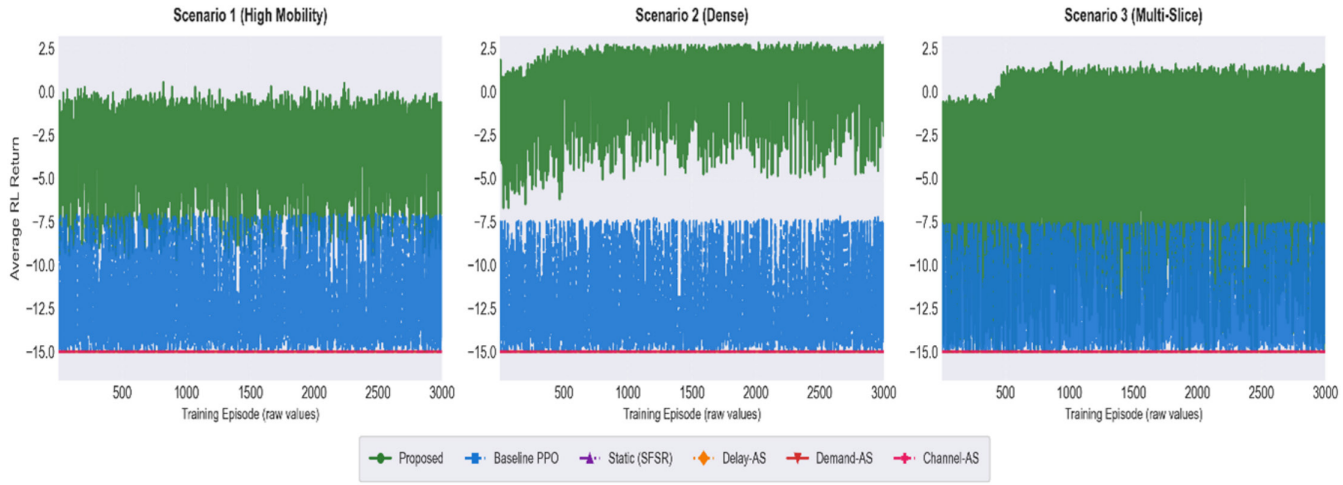


FIGURE 8 | Average RL return.

intervals. In contrast, the Baseline PPO exhibits larger SLA degradation during shock periods and more frequently falls below the target level. These results indicate that the proposed safety-aware control mechanism provides greater stability under sustained and abrupt traffic and mobility disturbances.

Figure 8 compares the RL-return evolution of the proposed framework and Baseline PPO. The proposed method converges to a higher average return with lower variability, indicating more stable reward optimization over the training process. Baseline PPO converges to a lower return and exhibits greater variability. The heuristic methods produce comparatively flat return profiles because their allocation rules are fixed and do not adapt through policy learning.

Figure 9 compares SLA performance during shock and non-shock episodes. The proposed method exhibits only a limited degradation between the two conditions, indicating robustness to the simulated traffic and mobility shocks, including longer shock periods with an average duration of 5.77 episodes. In contrast, Baseline PPO exhibits greater variability and a more pronounced reduction in SLA satisfaction during shock episodes. These results further indicate that the proposed framework is less sensitive to abrupt changes in traffic demand and mobility conditions.

Table 5 presents the comparison of the performance of some of the most important quantitative metrics under three representative scenarios.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

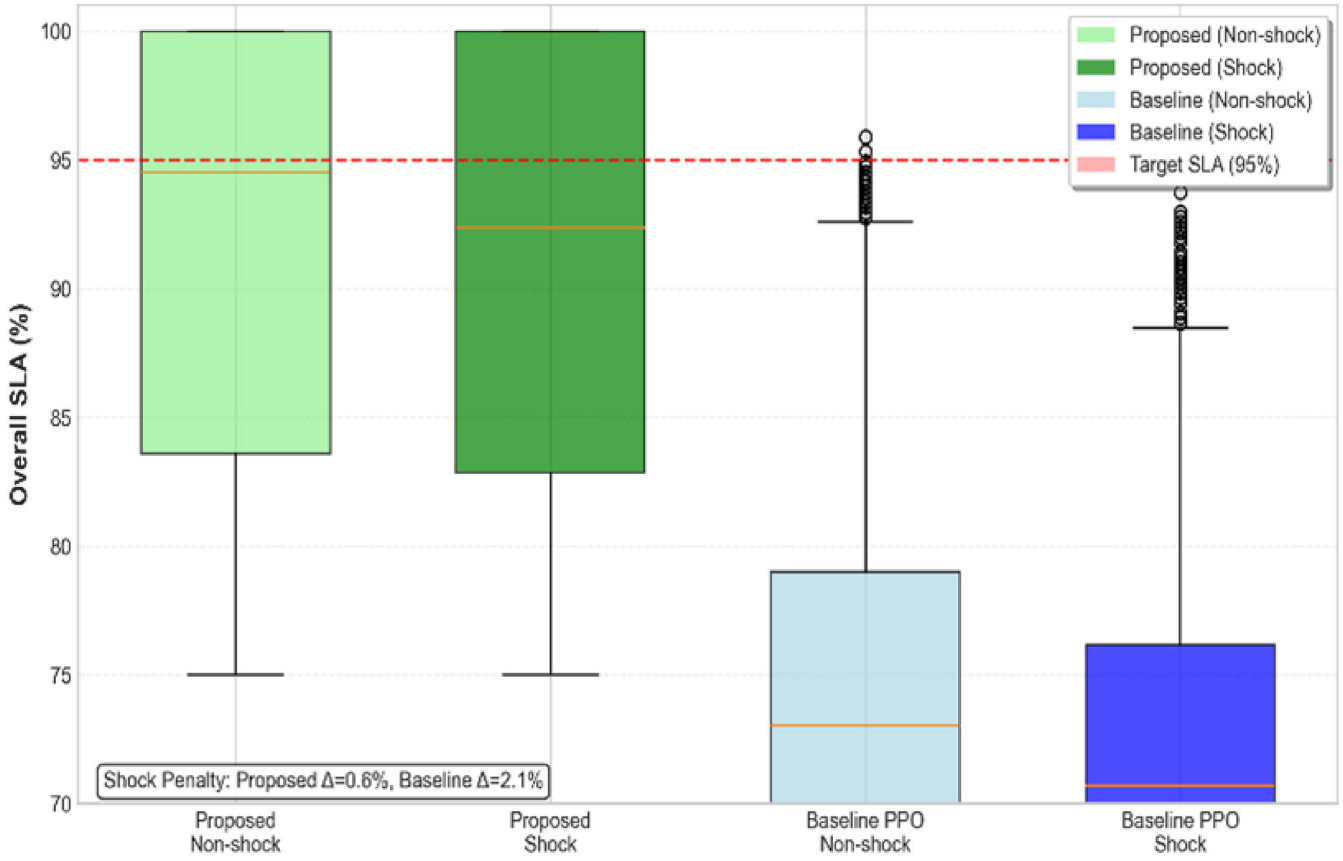


FIGURE 9 | SLA distribution shock vs. nonshock episodes.

a. *Average decision time (s):* The proposed approach achieves a consistent end-to-end controller decision time of 0.0038–0.0045 s across the three scenarios. This measurement includes state preparation, policy-network inference, safety-layer processing, and action generation, and remains well within the real-time operational requirement. Baseline PPO exhibits a slightly higher decision time of 0.0045–0.0060 s, particularly under dense and multislice conditions. Lagrangian-PPO shows a comparable but marginally higher decision time of 0.0048–0.0062 s because of the additional dual-variable update. TTS-DDQN requires approximately 0.0100–0.0114 s per decision, which is higher than the PPO-based methods because of its value-based decision process over the discrete action space. Heuristic schemes operate at lower decision times of 0.0029–0.0038 s owing to their fixed rule-based computation. Thus, although the proposed framework introduces adaptive learned control and safety-aware action processing, its decision-time overhead remains lower than TTS-DDQN and close to that of the simpler PPO-based controllers.

b. *P99 tail latency (ms):* The proposed framework achieves the lowest P99 tail latency, 4.24–4.53 ms, across the three scenarios, substantially lower than the 13.20–15.60-ms range observed for the heuristic schemes. Baseline PPO achieves 4.90–5.50 ms, whereas Lagrangian-PPO improves this slightly to 4.70–5.20 ms. TTS-DDQN records P99 latency of 7.22, 7.78, and 7.55 ms under the high-mobility, dense-deployment, and multislice scenarios, respectively. Although TTS-DDQN provides competitive overall SLA performance, its higher P99 latency indicates that value-based resource adaptation alone does not provide the explicit tail-latency regulation achieved by the proposed framework. The results therefore support the benefit of the proposed tail-latency-aware reward shaping and safety-aware control in suppressing severe latency events.

c. *RL policy variability (log-standard deviation):* The exploration–exploitation balance and convergence stability of the stochastic policy can be characterized through its log-standard deviation. The converged values of the proposed method are approximately -0.69, indicating controlled exploration and stable learning. Baseline PPO exhibits a higher value of approximately 0.1954, corresponding to broader policy stochasticity, whereas Lagrangian-PPO reduces this value to approximately 0.1200. The latter reflects narrower exploration under the additional constraint penalty, although its policy remains more stochastic than the converged proposed policy. Policy log-standard deviation is not applicable to TTS-DDQN because it is a value-based method using ϵ-greedy action selection rather than a Gaussian stochastic policy. Similarly, the heuristic schemes do not have a policy log-standard-deviation parameter because their allocation decisions are deterministic and rule based.

d. *Under-allocation index:* The under-allocation measure reflects the fraction of slice demand that is not served and is therefore associated with the risk of SLA degradation.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Baseline PPO exhibits moderate under-allocation of 22%–26% across the three scenarios, whereas Lagrangian-PPO reduces it to 17%–21%, indicating more direct control of resource shortages through the constraint penalty. TTS-DDQN exhibits considerably lower under-allocation, ranging from 0.92% to 1.69%. However, this result must be interpreted jointly with its over-allocation behavior. TTS-DDQN records approximately 59.99%–62.84% over-allocation across the three scenarios, indicating that its low under-allocation is achieved through comparatively aggressive resource provisioning.

The proposed framework reports under-allocation values in the range of 28.19%–58.41%; however, these values must likewise be interpreted jointly with the over-allocation and SLA results. Through its reservation-based control mechanism, the proposed framework preserves capacity headroom for safety-critical traffic and tail-latency protection rather than maximizing instantaneous demand coverage. Consequently, despite its higher reported under-allocation, the proposed framework achieves the highest overall SLA compliance and lowest tail latency while limiting over-allocation to only 1%–3%. This contrasts with TTS-DDQN, which achieves low under-allocation at the cost of approximately 60%–63% over-allocation, and with Lagrangian-PPO, which exhibits 43%–48% over-allocation. Several heuristic schemes also show low under-allocation because of their direct proportional or rule-based allocation behavior, but their SLA and tail-latency results show that low under-allocation alone does not imply effective service differentiation. Taken together, the under- and over-allocation results show that the proposed framework regulates resource allocation more conservatively while maintaining superior overall SLA and tail-latency performance.

e. *mMTC packet loss rate:* The proposed framework maintains low mMTC packet loss of 1.91%–2.09%, substantially below the 6.00% observed for the heuristic schemes, which exceeds the 5% SLA threshold. Baseline PPO achieves a lower loss rate of 1.25%, whereas Lagrangian-PPO achieves a comparable 1.22%–1.26%. TTS-DDQN records mMTC loss rates of 2.63%, 2.80%, and 2.71% for the high-mobility, dense-deployment, and multislice scenarios, respectively. Although these values are slightly higher than those of the proposed framework, they remain below the 5% mMTC loss threshold. The proposed method therefore maintains satisfactory mMTC reliability while simultaneously providing stronger overall SLA compliance and lower tail-latency than TTS-DDQN. Its marginally higher mMTC loss relative to Baseline PPO and Lagrangian-PPO reflects the cross-slice trade-off introduced by prioritizing URLLC tail protection and eMBB service continuity under the reservation and safety-control mechanisms.

Key performance variables frequently reported in recent network slicing research, such as those in [36], are covered in the proposed evaluation. The evaluation offers a thorough understanding of framework performance in multislice contexts by taking into account QoS satisfaction, learning stability, and resource consumption.

A one-at-a-time sensitivity analysis was carried out on the three main parameters listed in Section 4.6: initial BC weight w_0, SLA boost Δ_{max}, and history window H in order to evaluate the framework's resilience to BC scheduler adjustment. In the high-mobility scenario, each parameter was adjusted around its default value while keeping the others constant. The outcomes of the high mobility regime are shown in Table 6. URLLC SLA fluctuates between 62.1% and 67.2%, total SLA between 89.2% and 92.2%, and P95 latency between 4.4 and 5.0 ms throughout the swept range. Every metric is best serviced by the default configuration, with w_0 showing the biggest performance fluctuation (URLLC SLA range of 5.1 percentage points) and H showing the smallest. The framework maintains 89.2% overall SLA and 5.0-ms P95 latency even under the worst-case perturbation ($w_0 = 0.40$), demonstrating that performance is resilient to moderate scheduler parameter change within the swept range. Convergence behavior is consistent (980–1320 episodes), indicating that scheduler adjustment has a greater impact on steady-state performance than learning capacity.

To compare the Python surrogate against the high-fidelity ns-3 NR simulation, identical traffic conditions (high-mobility scenario, 30 UEs at 15 m/s, and equal-share allocation) were processed through both environments. Table 7 reports the resulting comparison in absolute terms. The surrogate closely tracks throughput (within 1.1 Mbps) and central-tendency latency statistics (within 4.64 ms on mean latency and 1.8 ms

TABLE 6 | BC scheduler sensitivity analysis.

Parameter	Value	URLLC SLA (%)	Overall SLA (%)	P95 latency (ms)	Episodes to converge
Initial BC weight (w_0)	0.40	62.1	89.2	5.0	1320
	0.60 (default)	67.2	92.2	4.5	980
	0.80	64.4	89.9	4.7	1280
SLA boost (Δ_{max})	0.10	62.8	89.9	4.9	1180
	0.35 (default)	67.2	92.2	4.5	980
History window (H)	5	65.9	91.7	4.4	1040
	10 (default)	67.2	92.2	4.5	980

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

on median latency), indicating that the surrogate's calibrated statistical abstraction captures typical-case behavior with reasonable fidelity. The gap widens at higher percentiles, reaching 8.5 ms at P90 and 28.19 ms at P95, which is consistent with the fact that the surrogate abstracts subframe PHY/MAC effects such as Rayleigh fast fading, HARQ retransmissions, and MAC scheduler contention into aggregate statistical couplings rather than simulating them directly. These effects can disproportionately influence tail behavior while having a smaller impact on typical-case latency. Both environments nonetheless agree qualitatively that equal-share allocation fails the 12-ms URLLC SLA threshold, confirming that the surrogate preserves the directional behavior of allocation policies even where absolute tail-latency values diverge. The BC stage of the proposed framework is trained on ns-3-generated NR traces that capture these PHY/MAC effects directly, providing the policy with implicit exposure to realistic network dynamics during initialization.

6.3 | Inference Latency Scalability

To assess inference scalability, the CNN–LSTM policy was benchmarked under progressively larger state and action dimensions while fixing the LSTM hidden dimension at $d = 128$. The resulting policy-network inference latencies are summarized in Table 8. These measurements represent the isolated forward-pass execution time and therefore exclude the state preparation, safety-layer processing, and action-generation overheads included in the end-to-end decision time reported in Table 5. Two axes were examined: gNB scaling ($M = 4$, $N = 4$ to $N = 50$) and slice scaling ($N = 50$, 500 UEs, $M = 4$ to $M = 10$). gNB scaling increased the state and action dimensions from 43/16 to 457/200, with mean and P95 inference latencies growing from 0.367/0.492 to 0.702/0.963 ms. Slice scaling further increased the dimensions to 763/500, with the largest configuration ($N = 50$, $M = 10$) yielding mean and P95 latencies of 3.667 and 3.925 ms, respectively—approximately 0.20% of the 2-s decision interval. These results demonstrate that online policy execution remains well within the 2-s control interval across all evaluated configurations and does not exhibit catastrophic inference-time degradation as the state and action dimensions increase.

State aggregation scales as $\mathcal{O}(NM)$ per decision epoch and CNN–LSTM inference scales as $\mathcal{O}(d^2)$ with fixed hidden dimension $d = 128$.

These measurements characterize online inference rather than training scalability. As PPO is an on-policy algorithm requiring fresh trajectories after each policy update, larger state and action spaces are expected to require additional training samples and longer convergence times because of the enlarged policy search space. Accordingly, the present study does not claim that the fixed 3000-episode training budget remains sufficient for substantially larger network topologies. For such deployments, hierarchical or multi-agent formulations can reduce centralized dimensionality: Partitioning N gNBs into G clusters reduces each local agent's action dimension from MN to approximately $M(N/G)$, whereas a per-gNB multi-agent design reduces it further to M, at the cost of coordination and nonstationarity overheads during training. These architectures provide a practical pathway for scaling the proposed framework to future large-scale beyond-5G and 6G RAN deployments.

Table 9 presents a closed-loop ns-3 validation of the best performing policy under the high-mobility scenario. Compared with the Python surrogate, the detailed PHY/MAC dynamics in

TABLE 7 | Surrogate-vs-ns-3 comparison under identical equal-share allocation (high-mobility scenario, 30 UEs at 15 m/s).

Metric	Surrogate	ns-3	Difference
Throughput (Mbps)	9.1	8.0	−1.1
Mean latency (ms)	12.16	16.8	+4.64
Median latency (ms)	11.4	13.2	+1.8
P90 latency (ms)	13.0	21.5	+8.5
P95 latency (ms)	13.81	42.0	+28.19
URLLC SLA (< 12 ms)	Failed	Failed	—

TABLE 8 | CNN–LSTM policy inference latency under increasing network scale. The 2-s decision interval corresponds to 2000 ms; all evaluated configurations satisfy this constraint.

Slices M	gNBs N	UEs	State dimension	Action dimension	Mean inference (ms)	Std. dev. (ms)	P95 inference (ms)	Within 2 s
4	4	80	43	16	0.367	0.118	0.492	Yes
4	8	160	79	32	0.349	0.069	0.495	Yes
4	16	320	151	64	0.373	0.083	0.490	Yes
4	32	500	295	128	0.544	0.111	0.708	Yes
4	50	500	457	200	0.702	0.160	0.963	Yes
6	50	500	559	300	1.970	0.707	2.800	Yes
8	50	500	661	400	2.755	0.234	3.075	Yes
10	50	500	763	500	3.667	0.258	3.925	Yes

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | Closed-loop ns-3 validation of the proposed policy under the high-mobility scenario.

Evaluation setting	P95 latency (ms)	URLLC SLA (%)	eMBB SLA (%)	Throughput (Mbps)
Proposed (Python surrogate)	4.50	67.2	100.0	9.40
Proposed (closed-loop ns-3)	11.70	45.0	94.0	8.25

ns-3 increase the P95 latency from 4.50 to 11.70 ms and reduce the URLLC SLA satisfaction from 67.2% to 45.0%, whereas the eMBB SLA remains high at 94.0%. Throughput decreases moderately from 9.40 to 8.25 Mbps. Compared with the equal-share ns-3 baseline reported in Table 7 (P95 latency of 42.0 ms), the proposed closed-loop policy maintains a substantially lower tail latency, demonstrating that the relative tail-latency improvement is preserved under higher fidelity PHY/MAC simulation despite the expected increase in absolute latency. The present closed-loop ns-3 evaluation provides surrogate-to-ns-3 transfer evidence for the evaluated high-mobility configuration, but it should not be interpreted as a topology-, channel-, or mobility-independent deployment guarantee. Broader validation under additional channel and mobility conditions, followed by physical testbed evaluation, is required to establish general sim-to-real applicability.

In addition to the sensitivity analysis of the CVaR threshold and traffic parameters, the influence of the reward-function weights was also investigated to examine how different service-priority settings affect multislice performance. Specifically, the URLLC reward weight was varied, whereas the remaining reward terms (efficiency, SLA, and utilization) were proportionally rescaled to preserve their original relative contributions and maintain a total reward weight of one.

Table 10 evaluates the sensitivity of the proposed framework to the URLLC reward weight. Increasing the URLLC reward weight from 0.40 to 0.70 improves URLLC SLA satisfaction from 58.3% to 71.4%, confirming that stronger emphasis on latency-sensitive traffic enhances URLLC reliability. However, further increasing the weight to 0.80 does not provide additional benefit, with URLLC SLA slightly decreasing to 70.1%, whereas eMBB SLA declines more noticeably from 96.6% to 91.3%. Consequently, the overall SLA decreases from 91.0% to 87.9%, indicating diminishing returns from excessive URLLC prioritization. This behavior is consistent with the sensitivity analysis in Table 3, where increasingly stringent reliability-related β-thresholds also reduced URLLC SLA, suggesting that overly aggressive optimization can lead to a less balanced control policy. Although $w_{\mathrm{URLLC}} = 0.70$ achieves the highest URLLC SLA, the default configuration ($w_{\mathrm{URLLC}} = 0.55$) achieves the highest overall SLA (92.2%), maintains perfect eMBB SLA satisfaction (100%), and therefore provides the best balance between latency-critical and throughput-oriented services for mixed-traffic network slicing deployments.

The low P95 latency and the URLLC SLA ratio capture different aspects of performance: P95 summarizes the tail-latency distribution, whereas SLA satisfaction reflects compliance across evaluation intervals. Hence, transient contention, mobility variations, and traffic shocks can still produce SLA violations despite a low aggregate P95. The results also indicate that substantially stronger URLLC guarantees may require hard latency constraints or hierarchical prioritization, potentially at the expense of competing slices. Although over-allocation reflects excess resource provisioning, it does not directly measure radio or processing energy consumption. Since the present evaluation does not include an explicit energy model, energy-aware SLA and tail-latency optimization is left for future work.

TABLE 10 | Sensitivity of the proposed framework to the URLLC reward weight. The remaining reward weights are proportionally rescaled while preserving their original relative contributions.

w_{URLLC}	URLLC SLA (%)	eMBB SLA (%)	Overall SLA (%)	Remark
0.40	58.3	99.4	89.1	Lower URLLC priority
0.55	67.2	100.0	92.2	Balanced (default)
0.70	71.4	96.6	91.0	Higher URLLC priority
0.80	70.1	91.3	87.9	Excessive URLLC weighting

7 | Conclusion

In this work, a dynamic framework of RAN slicing was proposed and proved to operate in the environment of heterogeneous traffic, strict constraints of the SLA, user mobility and with the primary emphasis on the reduction of URLLC tail-latency violations. Unlike the conventional methods, which maximize the mean performance, the suggested framework directly addresses the latency outliers that are infrequent but critical and lead to the deterioration of service in slices with safety concerns. By merging offline, trace driven BC with online policy optimization, the framework facilitates steady continuous action resource distribution in highly nonstationary RAN environments. Key design elements including tail-latency oriented reward shaping, bounded reliability-driven exploration control and safety aware reservation with emergency reallocation allow rapid corrective responses to SLA violations while preserving long term adaptability. Pretraining with realistic ns-3 NR traces significantly reduces unsafe exploration during early learning and SLA critical operating regimes. Extensive evaluations show that the proposed framework improves overall SLA satisfaction and URLLC tail-latency performance over the evaluated

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

learning-based and heuristic baselines. Closed-loop ns-3 validation further confirms that the tail-latency advantage is retained under higher fidelity PHY/MAC dynamics, although absolute performance degrades relative to the surrogate. Future work will consider broader channel and mobility conditions, larger scale hierarchical or multi-agent extensions, energy-aware optimization and physical testbed validation.

References

1. W. Wu, C. Zhou, M. Li, et al., "AI-Native Network Slicing for 6G Networks," *IEEE Wireless Communications* 29, no. 1 (2022): 96–103.

2. J. A. Hurtado Sánchez, K. Casilimas, and O. M. Caicedo Rendón, "Deep Reinforcement Learning for Resource Management on Network Slicing: A Survey," *Sensors* 22 (2022): 3031, https://doi.org/10.3390/s22083031.

3. M. Zangooei, N. Saha, M. Golkarifard, and R. Boutaba, "Reinforcement Learning for Radio Resource Management in RAN Slicing: A Survey," *IEEE Communications Magazine* 61, no. 2 (2023): 118–124, https://doi.org/10.1109/MCOM.004.2200532.

4. Z. Sasan and S. Khorsandi, "Balancing Resource Utilization and Slice Dissatisfaction Through Dynamic Soft Slicing for 6G Wireless Networks," *Scientific Reports* 15 (2025): 22987, https://doi.org/10.1038/s41598-025-06521-9.

5. Z. Wang, H. Li, L. Sun, T. Rosenkrantz, H. Che, and H. Jiang, "TailGuard: Tail Latency SLO Guaranteed Task Scheduling for Data-Intensive User-Facing Applications," in *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)* (IEEE, 2023), 898–909, https://doi.org/10.1109/ICDCS57875.2023.00042.

6. D. A. Chekired, M. A. Togou, L. Khoukhi, and A. Ksentini, "5G-Slicing-Enabled Scalable SDN Core Network: Toward an Ultra-Low Latency of Autonomous Driving Service," *IEEE Journal on Selected Areas in Communications* 37, no. 8 (2019): 1769–1782, https://doi.org/10.1109/JSAC.2019.2927065.

7. B. Lu, Y. Wu, L. Qian, S. Zhou, H. Zhang, and R. Lu, "Multi-Agent DRL-Based Two-Timescale Resource Allocation for Network Slicing in V2X Communications," *IEEE Transactions on Network and Service Management* 21, no. 6 (2024): 6744–6758, https://doi.org/10.1109/TNSM.2024.3454758.

8. J. Dai, L. Li, R. Safavinejad, et al., "O-RAN-Enabled Intelligent Network Slicing to Meet Service-Level Agreement (SLA)," *IEEE Transactions on Mobile Computing* 24, no. 2 (2025): 890–906, https://doi.org/10.1109/TMC.2024.3476338.

9. C. V. Nahum, V. H. L. Lopes, R. M. Dreifuerst, et al., "Intent-Aware Radio Resource Scheduling in a RAN Slicing Scenario Using Reinforcement Learning," *IEEE Transactions on Wireless Communications* 23, no. 3 (2024): 2253–2267, https://doi.org/10.1109/TWC.2023.3297014.

10. Y. Kim and H. Lim, "Multi-Agent Reinforcement Learning-Based Resource Management for End-to-End Network Slicing," *IEEE Access* 9 (2021): 56178–56190, https://doi.org/10.1109/ACCESS.2021.3072435.

11. J. Shokhanda, U. Pal, A. Kumar, S. Chattopadhyay, and A. Bhattacharya, "SafeTail: Tail Latency Optimization in Edge Service Scheduling via Redundancy Management," *IEEE Transactions on Network and Service Management* 22, no. 5 (2025): 4668–4680, https://doi.org/10.1109/TNSM.2025.3587752.

12. S. Choi, S. Choi, G. Lee, S.-G. Yoon, and S. Bahk, "Deep Reinforcement Learning for Scalable Dynamic Bandwidth Allocation in RAN Slicing With Highly Mobile Users," *IEEE Transactions on Vehicular Technology* 73, no. 1 (2024): 576–590, https://doi.org/10.1109/TVT.2023.3302416.

13. A. Nassar and Y. Yilmaz, "Deep Reinforcement Learning for Adaptive Network Slicing in 5G for Intelligent Vehicular Systems and Smart Cities," *IEEE Internet of Things Journal* 9 (2022): 222–235.

14. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

15. N. Wang, Z. Li, X. Liang, Y. Hou, and A. Yang, "A Review of Deep Reinforcement Learning Methods and Military Application Research," *Mathematical Problems in Engineering* 2023, no. 1 (2023): 7678382.

16. Y. Abiko, T. Saito, D. Ikeda, K. Ohta, T. Mizuno, and H. Mineno, "Flexible Resource Block Allocation to Multiple Slices for Radio Access Network Slicing Using Deep Reinforcement Learning," *IEEE Access* 8 (2020): 68183–68198.

17. Y. Wang, N. Liu, Z. Pan, and X. You, "AI-Based Resource Allocation in E2E Network Slicing With Both Public and Non-Public Slices," *Applied Sciences* 13, no. 22 (2023): 12505, https://doi.org/10.3390/app132212505.

18. A. Filali, Z. Mlika, S. Cherkaoui, and A. Kobbane, "Dynamic SDN-Based Radio Access Network Slicing With Deep Reinforcement Learning for URLLC and eMBB Services," *IEEE Transactions on Network Science and Engineering* 9, no. 4 (2022): 2174–2187.

19. Y. Jia, C. Zhang, N. Li, Y. Huang, and T. Q. S. Quek, "Hierarchical Intelligence Enabled Joint RAN Slicing and MAC Scheduling for SLA Guarantee," *IEEE Transactions on Communications* 73, no. 8 (2025): 6081–6094, https://doi.org/10.1109/TCOMM.2024.3519550.

20. J. Cai, P. Cheng, Z. Chen, M. Ding, B. Vucetic, and Y. Li, "Deep Reinforcement Learning for Online Resource Allocation in Network Slicing," *IEEE Transactions on Mobile Computing* 23, no. 6 (2024): 7099–7116, https://doi.org/10.1109/TMC.2023.3328950.

21. T. Wang, S. Chen, Y. Zhu, A. Tang, and X. Wang, "LinkSlice: Fine-Grained Network Slice Enforcement Based on Deep Reinforcement Learning," *IEEE Journal on Selected Areas in Communications* 40, no. 8 (2022): 2378–2394, https://doi.org/10.1109/JSAC.2022.3180776.

22. Z. Chen, J. Zhang, G. Min, Z. Ning, and J. Li, "Traffic-Aware Lightweight Hierarchical Offloading Toward Adaptive Slicing-Enabled SAGIN," *IEEE Journal on Selected Areas in Communications* 42, no. 12 (2024): 3536–3550, https://doi.org/10.1109/JSAC.2024.3459020.

23. Z. Z. Saleh, M. F. Abbod, and R. Nilavalan, "Intelligent Resource Allocation via Hybrid Reinforcement Learning in 5G Network Slicing," *IEEE Access* 13 (2025): 47440–47458, https://doi.org/10.1109/ACCESS.2025.3550518.

24. M. A. Habibi, B. Han, A. Fellan, et al., "Toward an Open, Intelligent, and End-to-End Architectural Framework for Network Slicing in 6G Communication Systems," *IEEE Open Journal of the Communications Society* 4 (2023): 1615–1658, https://doi.org/10.1109/OJCOMS.2023.3294445.

25. S. Xiong, Y. Huang, S. He, and C. Zhang, "Enhancing Radio Resource Management in RAN Slicing by Diffusion Model and Digital Twin," *IEEE Transactions on Communications* 73, no. 12 (2025): 15295–15311, https://doi.org/10.1109/TCOMM.2025.3594784.

26. Y. Hua, R. Li, Z. Zhao, X. Chen, and H. Zhang, "GAN-Powered Deep Distributional Reinforcement Learning for Resource Management in Network Slicing," *IEEE Journal on Selected Areas in Communications* 38 (2020): 334–349.

27. R. Li, C. Wang, Z. Zhao, R. Guo, and H. Zhang, "The LSTM-Based Advantage Actor-Critic Learning for Resource Management in Network Slicing With User Mobility," *IEEE Communications Letters* 24 (2020): 2005–2009.

28. Z. Zhang, Q. Wu, P. Fan, N. Cheng, W. Chen, and K. B. Letaief, "DRL-Based Optimization for AoI and Energy Consumption in C-V2X Enabled IoV," *IEEE Transactions on Green Communications and Networking* 9, no. 4 (2025): 2144–2159, https://doi.org/10.1109/TGCN.2025.3531902.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

29. Y. Xie, Q. Wu, P. Fan, et al., "Resource Allocation for Twin Maintenance and Task Processing in Vehicular Edge Computing Network," *IEEE Internet of Things Journal* 12, no. 15 (2025): 32008–32020, https://doi.org/10.1109/JIOT.2025.3576582.

30. X. Gu, Q. Wu, P. Fan, et al., "DRL-Based Resource Allocation for Motion Blur Resistant Federated Self-Supervised Learning in IoV," *IEEE Internet of Things Journal* 12, no. 6 (2025): 7067–7085, https://doi.org/10.1109/JIOT.2024.3492326.

31. A. Montazerolghaem, "Energy-Efficient Software-Defined 5G/6G Multimedia IoV: PID Controller-Based Approach," *IEEE Transactions on Vehicular Technology* 75, no. 6 (2026): 11621–11634, https://doi.org/10.1109/TVT.2025.3649936.

32. A. Montazerolghaem, "R^{2+1}: Integrated Vehicle Routing, Packet Routing, and Resource Allocation for Knowledge-Defined 5G/6G Multimedia Internet of Vehicles," *Vehicular Communications* 60 (2026): 101054, https://doi.org/10.1016/j.vehcom.2026.101054.

33. M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi, "A Survey of Imitation Learning: Algorithms, Recent Developments, and Challenges," *IEEE Transactions on Cybernetics* 54, no. 12 (2024): 7173–7186, https://doi.org/10.1109/TCYB.2024.3395626.

34. A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation Learning: A Survey of Learning Methods," *ACM Computing Surveys* 50, no. 2 (2018): 21, https://doi.org/10.1145/3054912.

35. Z. Tao, W. Xu, and X. You, "Digital Twin-Accelerated Online Deep Reinforcement Learning for Admission Control in Sliced Communication Networks," *IEEE Transactions on Communications* 73, no. 4 (2025): 2490–2504, https://doi.org/10.1109/TCOMM.2024.3476430.

36. P. B. Metre and G. Kalnoor, "A Comparative Study of Network Slicing Techniques for Effective Utilization of Channel for 5G and Beyond 5G Networks," *IEEE Access* 13 (2025): 31870–31888, https://doi.org/10.1109/ACCESS.2025.3542569.

10991131, 2026, 15, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/dac.70605 by Beijing University Post And, Wiley Online Library on [28/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License